

2026-05-05-p1-f06-mcp-full-lifecycle

P1-F06 — MCP Full Lifecycle Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Add client-side MCP capability to HelixCode: connect to external MCP servers via stdio, HTTP, SSE, WebSocket transports with OAuth 2.0 (RFC 8414 + PKCE); register external tools as agent-callable; expose `helixcode mcp {add,list,remove,test,auth,logs}` CLI and `/mcp` slash command.

Architecture: Extend `internal/mcp/` (currently server-side only) with client-side files alongside `server.go`. Reuse exported `MCPMessage/MCPError/Tool` types. Single Transport interface; one file per transport. Client per server holds state machine + pending RPC bookkeeping. Manager aggregates clients and exposes tools to `internal/tools/registry.go`. YAML is source of truth; CLI mutations round-trip the YAML.

Tech Stack: Go 1.26, testify v1.11, github.com/spf13/cobra v1.8, gopkg.in/yaml.v3 (already in go.mod), gorilla/websocket (already in go.mod), golang.org/x/oauth2 (already in go.mod), golang.org/x/sys/windows for Windows job objects. **No new external dependencies.** Standard-library `os/exec`, `net/http`, `crypto/sha256`, `encoding/base64`, `encoding/json`, `time`, `sync`, `sync/atomic`.

Spec: `docs/superpowers/specs/2026-05-05-p1-f06-mcp-full-lifecycle-design.md` (commit 386a4af)

Working directory for all go commands: `helix_code/` (the inner Go module). Git commands run from the meta-repo root `/run/media/milosvasic/DATA4TB/Projects/helix_code/` per the F01-F05 convention.

Anti-bluff smoke (run on every commit, FULL pattern):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo "clean"
```

Task list

☐

P1-F06-T01 — bootstrap evidence + advance PROGRESS

☐

P1-F06-T02 — `types.go` + `transport.go` interface + `BackoffSchedule` (TDD)

☐

P1-F06-T03 — `transport_stdio.go` + cross-platform unix/windows files (TDD)



P1-F06-T04 — transport_http.go with OAuth bearer header (TDD)



P1-F06-T05 — transport_sse.go with reconnect loop (TDD)



P1-F06-T06 — transport_ws.go via gorilla/websocket (TDD)



P1-F06-T07 — oauth.go: RFC 8414 discovery + PKCE + token cache (TDD)



P1-F06-T08 — lifecycle.go: Client state machine + handshake (TDD)



P1-F06-T09 — registry.go: Manager + tool merging (TDD)



P1-F06-T10 — config.go: YAML loader/saver, project + user precedence (TDD)



P1-F06-T11 — cmd/cli/mcp_cmd.go + /mcp slash command (TDD)



P1-F06-T12 — cmd/cli/main.go startup wiring + tools/registry.go integration + integration tests (no mocks)



P1-F06-T13 — Challenge with runtime evidence + cross-compile check



P1-F06-T14 — Feature 6 close-out + push to 4 remotes

Task 1: Bootstrap evidence + advance PROGRESS

Files: - Modify: docs/improvements/06_phase_1_evidence.md - Modify: docs/improvements/PROGRESS.md



Step 1: Append F06 section header to evidence file

Append (do NOT overwrite) to docs/improvements/06_phase_1_evidence.md:

P1-F06 — MCP Full Lifecycle (4 Transports + OAuth)

****Spec:**** ``docs/superpowers/specs/2026-05-05-p1-f06-mcp-full-lifecycle-design.md` (commit `386a4af`)`

****Plan:**** ``docs/superpowers/plans/2026-05-05-p1-f06-mcp-full-lifecycle.md``

****Started:**** 2026-05-05

****Status:**** active

Task evidence trail

(filled in commit-by-commit as tasks land)



Step 2: Update PROGRESS.md current focus block

Replace the existing “## Current focus” block with:

Current focus

- ****Active phase:**** P1 – claude-code feature porting
- ****Active feature:**** F06 – MCP Full Lifecycle (4 Transports + OAuth)
- ****Active task:**** P1-F06-T01 – bootstrap evidence + advance PROGRESS
- ****Last completed:**** P1-F05-T13 – Feature 5 (Hook-Based Extensibility) close-out + push
- ****Owner:**** agent (Claude Opus 4.7)
- ****Started:**** 2026-05-04
- ****Last touched:**** 2026-05-05
- ****Blocked-on:**** none



Step 3: Add F06 task list block to PROGRESS.md

After the existing F05 task list block (all 13 items checked), insert:

Active feature task list (P1-F06: MCP Full Lifecycle)

- [] P1-F06-T01 – bootstrap evidence + advance PROGRESS
- [] P1-F06-T02 – types.go + transport.go interface + BackoffSchedule (TDD)
- [] P1-F06-T03 – transport_stdio.go + cross-platform unix/windows files (TDD)
- [] P1-F06-T04 – transport_http.go with OAuth bearer header (TDD)
- [] P1-F06-T05 – transport_sse.go with reconnect loop (TDD)
- [] P1-F06-T06 – transport_ws.go via gorilla/websocket (TDD)
- [] P1-F06-T07 – oauth.go: RFC 8414 discovery + PKCE + token cache (TDD)
- [] P1-F06-T08 – lifecycle.go: Client state machine + handshake (TDD)
- [] P1-F06-T09 – registry.go: Manager + tool merging (TDD)
- [] P1-F06-T10 – config.go: YAML loader/saver, project + user precedence (TDD)
- [] P1-F06-T11 – cmd/cli/mcp_cmd.go + /mcp slash command (TDD)
- [] P1-F06-T12 – cmd/cli/main.go startup wiring + tools/registry.go integration + integration tests

- [] P1-F06-T13 – Challenge with runtime evidence + cross-compile check
- [] P1-F06-T14 – Feature 6 close-out + push



Step 4: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
    improvements/06_phase_1_evidence.md docs/improvements/
    PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
docs(P1-F06-T01): bootstrap Phase 1 / Feature 6 evidence +
    advance PROGRESS
```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

EOF
)"

Task 2: types.go + transport.go interface + BackoffSchedule (TDD)

Files: - Create: helix_code/internal/mcp/client_doc.go (package overview comment for client surface) - Create: helix_code/internal/mcp/types.go (TransportType, error vars, ClientState, Event) - Create: helix_code/internal/mcp/transport.go (Transport interface, BackoffSchedule) - Create: helix_code/internal/mcp/transport_test.go



Step 1: Write failing test for BackoffSchedule

Create helix_code/internal/mcp/transport_test.go:

```
package mcp

import (
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestBackoffSchedule_Sequence(t *testing.T) {
    bs := NewBackoffSchedule()
```

```

want := []time.Duration{1 * time.Second, 2 * time.Second, 4
    * time.Second, 8 * time.Second, 16 * time.Second, 30 *
    time.Second, 30 * time.Second}
for i, base := range want {
    got := bs.Next()
    // jitter ±20% means got ∈ [base*0.8, base*1.2]
    require.GreaterOrEqual(t, got,
        time.Duration(float64(base)*0.8), "step %d", i)
    require.LessOrEqual(t, got,
        time.Duration(float64(base)*1.2), "step %d", i)
}
}

```

```

func TestBackoffSchedule_ResetAfterSuccess(t *testing.T) {
    bs := NewBackoffSchedule()
    bs.Next()
    bs.Next()
    bs.Next()
    bs.Reset()
    got := bs.Next()
    assert.GreaterOrEqual(t, got,
        time.Duration(float64(time.Second)*0.8))
    assert.LessOrEqual(t, got,
        time.Duration(float64(time.Second)*1.2))
}

```

```

func TestTransportType_Validate(t *testing.T) {
    cases := map[TransportType]bool{
        TransportStdio: true,
        TransportHTTP:  true,
        TransportSSE:   true,
        TransportWS:    true,
        TransportType("bogus"): false,
        TransportType(""):      false,
    }
    for tt, ok := range cases {
        err := tt.Validate()
        if ok {
            assert.NoError(t, err, string(tt))
        } else {
            assert.Error(t, err, string(tt))
        }
    }
}

```

```

func TestClientState_String(t *testing.T) {
    assert.Equal(t, "disconnected", StateDisconnected.String())
    assert.Equal(t, "ready", StateReady.String())
}

```

```

    assert.Equal(t, "reconnecting", StateReconnecting.String())
    assert.Equal(t, "closed", StateClosed.String())
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestBackoffSchedule|
    TestTransportType|TestClientState" ./internal/mcp/...

```

Expected: FAIL with undefined: NewBackoffSchedule, TransportStdio, etc.



Step 3: Write client_doc.go

Create helix_code/internal/mcp/client_doc.go:

```

// Package mcp also provides client-side support for the Model
// Context Protocol.
//
// The client surface lets HelixCode connect to external MCP
// servers across
// four transports (stdio, HTTP, SSE, WebSocket) and expose their
// tools to
// the agent. The client surface is orthogonal to the existing
// server-side
// MCPServer/MCPSession types in this package – they share JSON-
// RPC framing
// (MCPMessage, MCPError) but never call each other.
//
// Entry points:
// - Manager: aggregates Clients across configured servers;
//   consumed by
//   internal/tools/registry.go to register external tools.
// - Client: one per server; owns a Transport and a state
//   machine.
// - Transport: abstracts stdio/HTTP/SSE/WS; one file per
//   transport.
//
// Configuration is YAML-first (.helixcode/mcp.yml in project +
// user dirs).
// CLI commands (helixcode mcp add/remove) round-trip the YAML.
package mcp

```



Step 4: Write types.go

Create helix_code/internal/mcp/types.go:

```

package mcp

import (
    "errors"
    "fmt"
    "sync/atomic"
)

// TransportType enumerates supported client-side transports.
type TransportType string

const (
    TransportStdio TransportType = "stdio"
    TransportHTTP   TransportType = "http"
    TransportSSE    TransportType = "sse"
    TransportWS     TransportType = "ws"
)

func (t TransportType) Validate() error {
    switch t {
    case TransportStdio, TransportHTTP, TransportSSE,
        TransportWS:
        return nil
    default:
        return fmt.Errorf("mcp: invalid transport %q (want stdio|http|sse|ws)", string(t))
    }
}

// ClientState is the high-level lifecycle state for a client
// connection.
type ClientState int32

const (
    StateDisconnected ClientState = iota
    StateConnecting
    StateInitializing
    StateReady
    StateReconnecting
    StateClosed
)

func (s ClientState) String() string {
    switch s {
    case StateDisconnected:
        return "disconnected"
    case StateConnecting:
        return "connecting"
    }
}

```

```

    case StateInitializing:
        return "initializing"
    case StateReady:
        return "ready"
    case StateReconnecting:
        return "reconnecting"
    case StateClosed:
        return "closed"
    default:
        return fmt.Sprintf("unknown(%d)", int32(s))
}
}

// LoadState atomically reads the state from a *atomic.Int32
// backing store.
func LoadState(p *atomic.Int32) ClientState {
    return ClientState(p.Load())
}

// StoreState atomically stores the state.
func StoreState(p *atomic.Int32, s ClientState) {
    p.Store(int32(s))
}

// Event represents a lifecycle event emitted by a Client.
type Event struct {
    Server string
    State  ClientState
    Err    error
    Msg    string
}

// Client-side error sentinels.
var (
    ErrServerNotFound = errors.New("mcp: server not found")
    ErrNotReady       = errors.New("mcp: client not ready")
    ErrReconnect      = errors.New("mcp: transport
        reconnecting")
    ErrInitFailed     = errors.New("mcp: initialize handshake
        failed")
    ErrTransportClosed = errors.New("mcp: transport closed")
    ErrOAuthRequired  = errors.New("mcp: oauth token missing or
        invalid; run 'helixcode mcp auth'")
    ErrToolNotFound   = errors.New("mcp: tool not found on
        server")
    ErrProtocol       = errors.New("mcp: protocol violation")
)

```

```

    ErrTooManyPending = errors.New("mcp: too many pending
        requests")
)

```



Step 5: Write transport.go

Create helix_code/internal/mcp/transport.go:

```

package mcp

import (
    "context"
    "math/rand"
    "sync"
    "time"
)

// Transport is the seam every MCP client transport plugs into.
// Implementations must be safe to use from a single Recv
// goroutine and a
// concurrent Send caller. Close MUST be idempotent and unblock
// any pending
// Recv with io.EOF or ErrTransportClosed.
type Transport interface {
    Open(ctx context.Context) error
    Send(ctx context.Context, msg *MCPMessage) error
    Recv(ctx context.Context) (*MCPMessage, error)
    Close() error
    Type() TransportType
}

// BackoffSchedule produces exponentially increasing delays with
// ±20% jitter,
// capped at 30s. Reset() returns to the 1s base.
type BackoffSchedule struct {
    mu      sync.Mutex
    steps []time.Duration
    idx     int
    rng     *rand.Rand
}

func NewBackoffSchedule() *BackoffSchedule {
    return &BackoffSchedule{
        steps: []time.Duration{
            1 * time.Second,
            2 * time.Second,
            4 * time.Second,
            8 * time.Second,

```

```

        16 * time.Second,
        30 * time.Second,
    },
    rng: rand.New(rand.NewSource(time.Now().UnixNano())),
}

func (b *BackoffSchedule) Next() time.Duration {
    b.mu.Lock()
    defer b.mu.Unlock()
    base := b.steps[b.idx]
    if b.idx < len(b.steps)-1 {
        b.idx++
    }
    jitter := 0.8 + 0.4*b.rng.Float64() // [0.8, 1.2)
    return time.Duration(float64(base) * jitter)
}

func (b *BackoffSchedule) Reset() {
    b.mu.Lock()
    b.idx = 0
    b.mu.Unlock()
}

```



Step 6: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -run "TestBackoffSchedule|
    TestTransportType|TestClientState" ./internal/mcp/...

```

Expected: PASS (4/4).



Step 7: Anti-bluff smoke

```

cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
    placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
    "clean"

```

Expected: clean.



Step 8: Commit

```

git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/client_doc.go helix_code/
    internal/mcp/types.go helix_code/internal/mcp/
    transport.go helix_code/internal/mcp/transport_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'

```

```
feat(P1-F06-T02): add MCP client types + Transport interface +  
BackoffSchedule
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)  
<noreply@anthropic.com>
```

```
EOF
```

```
)"
```

Task 3: transport_stdio.go + cross-platform unix/windows files (TDD)

Files: - Create: helix_code/internal/mcp/transport_stdio.go - Create: helix_code/internal/mcp/transport_stdio_unix.go (//go:build unix) - Create: helix_code/internal/mcp/transport_stdio_windows.go (//go:build windows) - Create: helix_code/internal/mcp/transport_stdio_test.go - Create: helix_code/internal/mcp/testhelper_echo_server/main.go (Go test helper that echoes JSON-RPC over stdin/stdout)



Step 1: Write the test helper subprocess

Create helix_code/internal/mcp/testhelper_echo_server/main.go:

```
// echo MCP server: reads newline-delimited JSON-RPC from stdin,  
// replies with  
// either an empty result (request) or echoes notifications.  
// Writes a banner  
// to stderr so stderr-capture tests can assert on it. Terminates  
// on EOF.
```

```
package main
```

```
import (  
    "bufio"  
    "encoding/json"  
    "fmt"  
    "os"  
)
```

```
type rpc struct {  
    JSONRPC string `json:"jsonrpc"`  
    ID       *json.RawMessage `json:"id,omitempty"`  
    Method   string          `json:"method"`  
    Params   json.RawMessage `json:"params,omitempty"`  
    Result    json.RawMessage `json:"result,omitempty"`  
    Error     json.RawMessage `json:"error,omitempty"`  
}
```

```

func main() {
    fmt.Fprintln(os.Stderr, "echo-mcp-server: ready")
    in := bufio.NewScanner(os.Stdin)
    in.Buffer(make([]byte, 1024*1024), 16*1024*1024)
    out := bufio.NewWriter(os.Stdout)
    defer out.Flush()
    for in.Scan() {
        var req rpc
        if err := json.Unmarshal(in.Bytes(), &req); err != nil {
            fmt.Fprintf(os.Stderr, "parse error: %v\n", err)
            continue
        }
        if req.ID == nil {
            // notification: no reply
            fmt.Fprintf(os.Stderr, "notif: %s\n", req.Method)
            continue
        }
        // send empty-success reply
        resp := rpc{JSONRPC: "2.0", ID: req.ID, Result:
            json.RawMessage(`{}`)}
        b, _ := json.Marshal(&resp)
        out.Write(b)
        out.WriteByte('\n')
        out.Flush()
    }
}

```



Step 2: Write failing test

Create `helix_code/internal/mcp/transport_stdio_test.go`:

```

package mcp

import (
    "context"
    "os/exec"
    "path/filepath"
    "runtime"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func buildEchoServer(t *testing.T) string {
    t.Helper()
    dir := t.TempDir()

```

```

    bin := filepath.Join(dir, "echo-mcp-server")
    if runtime.GOOS == "windows" {
        bin += ".exe"
    }
    cmd := exec.Command("go", "build", "-o", bin, "./
        testhelper_echo_server")
    cmd.Dir = "."
    out, err := cmd.CombinedOutput()
    require.NoError(t, err, string(out))
    return bin
}

func TestStdioTransport_RoundTrip(t *testing.T) {
    bin := buildEchoServer(t)
    tr := NewStdioTransport(StdioConfig{
        Command: []string{bin},
    })
    ctx, cancel := context.WithTimeout(context.Background(),
        10*time.Second)
    defer cancel()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()

    id := "1"
    idRaw := []byte(`"1"`)
    require.NoError(t, tr.Send(ctx, &MCPMessage{
        JSONRPC: "2.0",
        ID:      id,
        Method:  "ping",
    })))
    _ = idRaw
    resp, err := tr.Recv(ctx)
    require.NoError(t, err)
    assert.Equal(t, "1", resp.ID)
    assert.Equal(t, TransportStdio, tr.Type())
}

func TestStdioTransport_StderrCapture(t *testing.T) {
    bin := buildEchoServer(t)
    tr := NewStdioTransport(StdioConfig{Command: []string{bin}})
    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
    defer cancel()
    require.NoError(t, tr.Open(ctx))
    // echo server writes "ready" banner to stderr at startup
    time.Sleep(200 * time.Millisecond)
    stderr := tr.Stderr()
    assert.Contains(t, string(stderr), "echo-mcp-server: ready")
}

```

```

        require.NoError(t, tr.Close())
    }

    func TestStdioTransport_CloseKillsProcess(t *testing.T) {
        bin := buildEchoServer(t)
        tr := NewStdioTransport(StdioConfig{Command: []string{bin}})
        ctx, cancel := context.WithTimeout(context.Background(),
            5*time.Second)
        defer cancel()
        require.NoError(t, tr.Open(ctx))
        pid := tr.PID()
        require.NotZero(t, pid)
        require.NoError(t, tr.Close())
        // after Close, sending must fail
        err := tr.Send(ctx, &MCPMessage{JSONRPC: "2.0", ID: "x",
            Method: "ping"})
        assert.Error(t, err)
    }

```



Step 3: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestStdioTransport" ./
    internal/mcp/...

```

Expected: FAIL with undefined: NewStdioTransport, StdioConfig.



Step 4: Write transport_stdio.go (cross-platform core)

Create helix_code/internal/mcp/transport_stdio.go:

```

package mcp

import (
    "bufio"
    "bytes"
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "io"
    "os/exec"
    "sync"

    // StdioConfig configures a stdio MCP server subprocess.
    type StdioConfig struct {
        Command []string // argv (Command[0] = executable)
        Env      map[string]string // additional env vars
    }

```

```

    Cwd      string           // working dir; empty = inherit
}

// stderrRing is a 64KB ring buffer for subprocess stderr.
type stderrRing struct {
    mu sync.Mutex
    buf bytes.Buffer
    cap int
}

func (r *stderrRing) Write(p []byte) (int, error) {
    r.mu.Lock()
    defer r.mu.Unlock()
    if r.buf.Len()+len(p) > r.cap {
        drop := r.buf.Len() + len(p) - r.cap
        if drop > r.buf.Len() {
            r.buf.Reset()
        } else {
            r.buf.Next(drop)
        }
    }
    return r.buf.Write(p)
}

func (r *stderrRing) Snapshot() []byte {
    r.mu.Lock()
    defer r.mu.Unlock()
    out := make([]byte, r.buf.Len())
    copy(out, r.buf.Bytes())
    return out
}

// stdioTransport runs an MCP server as a subprocess and frames
//      JSON-RPC over
//      newline-delimited stdin/stdout.
type stdioTransport struct {
    cfg      StdioConfig
    cmd      *exec.Cmd
    stdin    io.WriteCloser
    stdout   *bufio.Reader
    ring     *stderrRing
    mu       sync.Mutex
    closed   bool
    closeCh  chan struct{}
}

// NewStdioTransport returns a new stdio transport. Open spawns
//      the subprocess.

```

```

func NewStdioTransport(cfg StdioConfig) *stdioTransport {
    return &stdioTransport{
        cfg:      cfg,
        ring:      &stderrRing{cap: 64 * 1024},
        closeCh:   make(chan struct{}),
    }
}

func (t *stdioTransport) Type() TransportType { return
    TransportStdio }

func (t *stdioTransport) Open(ctx context.Context) error {
    if len(t.cfg.Command) == 0 {
        return fmt.Errorf("mcp stdio: empty command")
    }
    t.cmd = exec.CommandContext(ctx, t.cfg.Command[0],
        t.cfg.Command[1:]...)
    if t.cfg.Cwd != "" {
        t.cmd.Dir = t.cfg.Cwd
    }
    t.cmd.Env = mergeEnv(t.cfg.Env)
    configureProcAttrs(t.cmd) // platform-specific (unix/windows
        file)

    stdin, err := t.cmd.StdinPipe()
    if err != nil {
        return fmt.Errorf("mcp stdio: stdin pipe: %w", err)
    }
    stdout, err := t.cmd.StdoutPipe()
    if err != nil {
        return fmt.Errorf("mcp stdio: stdout pipe: %w", err)
    }
    stderr, err := t.cmd.StderrPipe()
    if err != nil {
        return fmt.Errorf("mcp stdio: stderr pipe: %w", err)
    }
    if err := t.cmd.Start(); err != nil {
        return fmt.Errorf("mcp stdio: start %v: %w",
            t.cfg.Command, err)
    }
    t.stdin = stdin
    t.stdout = bufio.NewReaderSize(stdout, 1024*1024)
    go func() {
        buf := make([]byte, 4096)
        for {
            n, err := stderr.Read(buf)
            if n > 0 {
                t.ring.Write(buf[:n])
            }
        }
    }()
}

```

```

        }
        if err != nil {
            return
        }
    }
}()
return nil
}

func (t *stdioTransport) Send(ctx context.Context, msg
    *MCPMessage) error {
    t.mu.Lock()
    defer t.mu.Unlock()
    if t.closed || t.stdin == nil {
        return ErrTransportClosed
    }
    b, err := json.Marshal(msg)
    if err != nil {
        return fmt.Errorf("mcp stdio: marshal: %w", err)
    }
    b = append(b, '\n')
    if _, err := t.stdin.Write(b); err != nil {
        return fmt.Errorf("mcp stdio: write: %w", err)
    }
    return nil
}

func (t *stdioTransport) Recv(ctx context.Context) (*MCPMessage,
    error) {
    if t.closed || t.stdout == nil {
        return nil, ErrTransportClosed
    }
    type result struct {
        msg *MCPMessage
        err error
    }
    ch := make(chan result, 1)
    go func() {
        line, err := t.stdout.ReadBytes('\n')
        if err != nil {
            if errors.Is(err, io.EOF) && len(line) == 0 {
                ch <- result{nil, io.EOF}
                return
            }
        }
    }
    var m MCPMessage
    if uerr := json.Unmarshal(bytes.TrimSpace(line), &m);
    uerr != nil {

```

```

        ch <- result{nil, fmt.Errorf("%w: %v", ErrProtocol,
uerr)}
        return
    }
    ch <- result{&m, err}
}()
select {
case <-ctx.Done():
    return nil, ctx.Err()
case r := <-ch:
    if r.err == io.EOF {
        return nil, io.EOF
    }
    if r.err != nil && r.msg == nil {
        return nil, r.err
    }
    return r.msg, nil
case <-t.closeCh:
    return nil, ErrTransportClosed
}
}

```

```

func (t *stdioTransport) Close() error {
    t.mu.Lock()
    if t.closed {
        t.mu.Unlock()
        return nil
    }
    t.closed = true
    close(t.closeCh)
    t.mu.Unlock()
    if t.stdin != nil {
        _ = t.stdin.Close()
    }
    if t.cmd != nil && t.cmd.Process != nil {
        _ = killProcessGroup(t.cmd) // platform-specific
        _, _ = t.cmd.Process.Wait()
    }
    return nil
}

```

// PID returns the subprocess PID (or 0 if not started). Test helper.

```

func (t *stdioTransport) PID() int {
    if t.cmd == nil || t.cmd.Process == nil {
        return 0
    }
    return t.cmd.Process.Pid
}

```

```

}

// Stderr returns a snapshot of the subprocess stderr ring
// buffer.
func (t *stdioTransport) Stderr() []byte {
    return t.ring.Snapshot()
}

func mergeEnv(extra map[string]string) []string {
    out := append([]string(nil), getEnv()...)
    for k, v := range extra {
        out = append(out, k+"="+v)
    }
    return out
}

```



Step 5: Write transport_stdio_unix.go

Create helix_code/internal/mcp/transport_stdio_unix.go:

```

//go:build unix

package mcp

import (
    "os"
    "os/exec"
    "syscall"
)

func configureProcAttrs(cmd *exec.Cmd) {
    cmd.SysProcAttr = &syscall.SysProcAttr{Setpgid: true}
}

// killProcessGroup sends SIGKILL to the process group of cmd's
// process.
func killProcessGroup(cmd *exec.Cmd) error {
    if cmd.Process == nil {
        return nil
    }
    pgid, err := syscall.Getpgid(cmd.Process.Pid)
    if err != nil {
        return cmd.Process.Kill()
    }
    return syscall.Kill(-pgid, syscall.SIGKILL)
}

func getEnv() []string {

```

```
    return os.Environ()
}
```



Step 6: Write transport_stdio_windows.go

Create helix_code/internal/mcp/transport_stdio_windows.go:

```
//go:build windows
```

```
package mcp
```

```
import (
    "os"
    "os/exec"
    "syscall"
)
```

```
func configureProcAttrs(cmd *exec.Cmd) {
    cmd.SysProcAttr = &syscall.SysProcAttr{
        CreationFlags: 0x000000200, // CREATE_NEW_PROCESS_GROUP
    }
}
```

```
// killProcessGroup terminates the subprocess; Windows does not
// have process
// groups in the unix sense, so we kill the direct child. For
// full job-object
// containment we would attach the child to a JobObject in
// configureProcAttrs;
// this implementation kills the child via Process.Kill which
// calls
// TerminateProcess, sufficient for the F06 surface
// (CREATE_NEW_PROCESS_GROUP
// already prevents Ctrl+C from propagating to our parent).
```

```
func killProcessGroup(cmd *exec.Cmd) error {
    if cmd.Process == nil {
        return nil
    }
    return cmd.Process.Kill()
}
```

```
func getEnv() []string {
    return os.Environ()
}
```



Step 7: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestStdioTransport" ./
internal/mcp/...
```

Expected: PASS (3/3).



Step 8: Cross-compile check (Linux + Windows)

```
cd HelixCode && go build ./internal/mcp/... && GOOS=windows go
build ./internal/mcp/...
```

Expected: both succeed without error.



Step 9: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 10: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
helix_code/internal/mcp/transport_stdio.go helix_code/
internal/mcp/transport_stdio_unix.go helix_code/internal/
mcp/transport_stdio_windows.go helix_code/internal/mcp/
transport_stdio_test.go helix_code/internal/mcp/
testhelper_echo_server/main.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
feat(P1-F06-T03): add stdio MCP transport with cross-platform
process group control
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```
EOF
)"
```

Task 4: transport_http.go with OAuth bearer header (TDD)

Files: - Create: helix_code/internal/mcp/transport_http.go - Create:
helix_code/internal/mcp/transport_http_test.go



Step 1: Write failing test

Create helix_code/internal/mcp/transport_http_test.go:

```

package mcp

import (
    "context"
    "encoding/json"
    "io"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "golang.org/x/oauth2"
)

func TestHTTPTransport_RoundTrip(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            assert.Equal(t, "POST", r.Method)
            assert.Equal(t, "application/json",
                r.Header.Get("Content-Type"))
            body, _ := io.ReadAll(r.Body)
            var req MCPMessage
            require.NoError(t, json.Unmarshal(body, &req))
            resp := MCPMessage{JSONRPC: "2.0", ID: req.ID, Result:
                map[string]any{"ok": true}}
            w.Header().Set("Content-Type", "application/json")
            _ = json.NewEncoder(w).Encode(&resp)
        }))
    defer srv.Close()

    tr := NewHTTPTransport(HTTPConfig{URL: srv.URL})
    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
    defer cancel()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()
    require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
        ID: "1", Method: "ping"}))
    resp, err := tr.Recv(ctx)
    require.NoError(t, err)
    assert.Equal(t, "1", resp.ID)
    assert.Equal(t, TransportHTTP, tr.Type())
}

func TestHTTPTransport_BearerHeader(t *testing.T) {

```

```

var seenAuth string
srv := httptest.NewServer(http.HandlerFunc(func(w
    http.ResponseWriter, r *http.Request) {
        seenAuth = r.Header.Get("Authorization")
        _ = json.NewEncoder(w).Encode(&MCPMessage{JSONRPC:
            "2.0", ID: "1", Result: map[string]any{}}}
    }))
defer srv.Close()
ts := oauth2.StaticTokenSource(&oauth2.Token{AccessToken:
    "tok-xyz"})
tr := NewHTTPTransport(HTTPConfig{URL: srv.URL, TokenSource:
    ts})
ctx := context.Background()
require.NoError(t, tr.Open(ctx))
defer tr.Close()
require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
    ID: "1", Method: "ping"}))
_, err := tr.Recv(ctx)
require.NoError(t, err)
assert.Equal(t, "Bearer tok-xyz", seenAuth)
}

func TestHTTPTransport_401WithOAuth(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            http.Error(w, "unauthorized", 401)
        }))
    defer srv.Close()
    ts := oauth2.StaticTokenSource(&oauth2.Token{AccessToken:
        "expired"})
    tr := NewHTTPTransport(HTTPConfig{URL: srv.URL, TokenSource:
        ts, OAuthEnabled: true})
    ctx := context.Background()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()
    require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
        ID: "1", Method: "ping"}))
    _, err := tr.Recv(ctx)
    require.ErrorIs(t, err, ErrOAuthRequired)
}

func TestHTTPTransport_4xxNoOAuth(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            http.Error(w, "bad request", 400)
        }))
    defer srv.Close()
    tr := NewHTTPTransport(HTTPConfig{URL: srv.URL})

```

```

    ctx := context.Background()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()
    require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
        ID: "1", Method: "ping"}))
    _, err := tr.Recv(ctx)
    require.Error(t, err)
    assert.True(t, strings.Contains(err.Error(), "400") ||
        isErrProtocol(err))
}

func isErrProtocol(err error) bool {
    for ; err != nil; err = unwrapErr(err) {
        if err == ErrProtocol {
            return true
        }
    }
    return false
}

func unwrapErr(e error) error {
    type unwrapper interface{ Unwrap() error }
    if u, ok := e.(unwrapper); ok {
        return u.Unwrap()
    }
    return nil
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestHTTPTransport" ./
    internal/mcp/...

```

Expected: FAIL with undefined: NewHTTPTransport, HTTPConfig.



Step 3: Write transport_http.go

Create helix_code/internal/mcp/transport_http.go:

```

package mcp

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "net/http"

```

```

    "sync"
    "time"

    "golang.org/x/oauth2"
)

type HTTPConfig struct {
    URL          string
    TokenSource   oauth2.TokenSource // nil = no auth
    OAuthEnabled bool           // determines 401 mapping
                  (ErrOAuthRequired vs ErrProtocol)
    Headers      map[string]string // extra static headers
    Timeout      time.Duration      // per-request; 0 = 60s
                  default
}

type httpTransport struct {
    cfg      HTTPConfig
    client    *http.Client
    recvCh    chan *recvItem
    mu        sync.Mutex
    closed    bool
}

type recvItem struct {
    msg *MCPMessage
    err error
}

func NewHTTPTransport(cfg HTTPConfig) *httpTransport {
    to := cfg.Timeout
    if to == 0 {
        to = 60 * time.Second
    }
    return &httpTransport{
        cfg:      cfg,
        client:   &http.Client{Timeout: to},
        recvCh:   make(chan *recvItem, 16),
    }
}

func (t *httpTransport) Type() TransportType { return TransportHTTP }

func (t *httpTransport) Open(ctx context.Context) error {
    if t.cfg.URL == "" {
        return fmt.Errorf("mcp http: empty URL")
    }
}

```

```

    return nil
}

func (t *httpTransport) Send(ctx context.Context, msg
    *MCPMessage) error {
    t.mu.Lock()
    if t.closed {
        t.mu.Unlock()
        return ErrTransportClosed
    }
    t.mu.Unlock()
    body, err := json.Marshal(msg)
    if err != nil {
        return fmt.Errorf("mcp http: marshal: %w", err)
    }
    go t.sendOne(ctx, body)
    return nil
}

func (t *httpTransport) sendOne(ctx context.Context, body
    []byte) {
    req, err := http.NewRequestWithContext(ctx, "POST",
        t.cfg.URL, bytes.NewReader(body))
    if err != nil {
        t.recvCh <- &recvItem{err: fmt.Errorf("mcp http: build
            request: %w", err)}
        return
    }
    req.Header.Set("Content-Type", "application/json")
    for k, v := range t.cfg.Headers {
        req.Header.Set(k, v)
    }
    if t.cfg.TokenSource != nil {
        tok, err := t.cfg.TokenSource.Token()
        if err != nil {
            t.recvCh <- &recvItem{err: fmt.Errorf("%w: %v",
                ErrOAuthRequired, err)}
            return
        }
        req.Header.Set("Authorization", tok.Type()+"
            "+tok.AccessToken)
        // oauth2.Token.Type() defaults to "Bearer" when empty
        if tok.Type() == "" {
            req.Header.Set("Authorization", "Bearer
                "+tok.AccessToken)
        }
    }
    resp, err := t.client.Do(req)

```

```

    if err != nil {
        t.recvCh <- &recvItem{err: fmt.Errorf("mcp http: do:
            %w", err)}
        return
    }
    defer resp.Body.Close()
    respBody, _ := io.ReadAll(io.LimitReader(resp.Body,
        32*1024*1024))
    if resp.StatusCode == 401 {
        if t.cfg.OAuthEnabled {
            t.recvCh <- &recvItem{err: ErrOAuthRequired}
        } else {
            t.recvCh <- &recvItem{err: fmt.Errorf("%w: 401 %s",
                ErrProtocol, string(respBody))}
        }
        return
    }
    if resp.StatusCode < 200 || resp.StatusCode >= 300 {
        t.recvCh <- &recvItem{err: fmt.Errorf("%w: status %d:
            %s", ErrProtocol, resp.StatusCode, string(respBody))}
        return
    }
    var m MCPMessage
    if err := json.Unmarshal(respBody, &m); err != nil {
        t.recvCh <- &recvItem{err: fmt.Errorf("%w: parse
            response: %v", ErrProtocol, err)}
        return
    }
    t.recvCh <- &recvItem{msg: &m}
}

func (t *httpTransport) Recv(ctx context.Context) (*MCPMessage,
    error) {
    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case item := <-t.recvCh:
        if item.err != nil {
            return nil, item.err
        }
        return item.msg, nil
    }
}

func (t *httpTransport) Close() error {
    t.mu.Lock()
    t.closed = true
    t.mu.Unlock()
}

```

```
    return nil
}
```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestHTTPTransport" ./
    internal/mcp/...
```

Expected: PASS (4/4).



Step 5: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
    "clean"
```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/transport_http.go helix_code/
    internal/mcp/transport_http_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T04): add HTTP MCP transport with OAuth bearer header

Co-Authored-By: Claude Opus 4.7 (1M context)
    <noreply@anthropic.com>
EOF
)"
```

Task 5: transport_sse.go with reconnect loop (TDD)

Files: - Create: helix_code/internal/mcp/transport_sse.go - Create:
helix_code/internal/mcp/transport_sse_test.go



Step 1: Write failing test

Create helix_code/internal/mcp/transport_sse_test.go:

```
package mcp

import (
    "context"
    "encoding/json"
```

```

    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "sync/atomic"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

// runSSEServer returns (postURL, sseURL, controlCloseStream,
//      cleanup).
// controlCloseStream() closes the active SSE stream so the
//      client must reconnect.
func runSSEServer(t *testing.T) (string, string, func(), func())
{
    t.Helper()
    mux := http.NewServeMux()
    var sessionID atomic.Int64
    type session struct {
        flusher http.Flusher
        w        http.ResponseWriter
        done     chan struct{}
    }
    var current atomic.Pointer[session]

    mux.HandleFunc("/sse", func(w http.ResponseWriter, r
        *http.Request) {
        flusher, ok := w.(http.Flusher)
        if !ok {
            http.Error(w, "no flush", 500)
            return
        }
        w.Header().Set("Content-Type", "text/event-stream")
        w.Header().Set("Cache-Control", "no-cache")
        flusher.Flush()
        s := &session{flusher: flusher, w: w, done: make(chan
            struct{})}
        current.Store(s)
        sessionID.Add(1)
        <-s.done
    })
    mux.HandleFunc("/post", func(w http.ResponseWriter, r
        *http.Request) {
        body, _ := io.ReadAll(r.Body)
        var req MCPMessage

```

```

        require.NoError(t, json.Unmarshal(body, &req))
        s := current.Load()
        if s == nil {
            http.Error(w, "no session", 503)
            return
        }
        resp := MCPMessage{JSONRPC: "2.0", ID: req.ID, Result:
            map[string]any{"ok": true}}
        b, _ := json.Marshal(&resp)
        fmt.Fprintf(s.w, "data: %s\n\n", string(b))
        s.flusher.Flush()
        w.WriteHeader(204)
    })
    srv := httptest.NewServer(mux)
    closeStream := func() {
        s := current.Load()
        if s != nil {
            close(s.done)
            current.Store(nil)
        }
    }
    cleanup := func() {
        closeStream()
        srv.Close()
    }
    return srv.URL + "/post", srv.URL + "/sse", closeStream,
        cleanup
}

func TestSSETransport_RoundTrip(t *testing.T) {
    postURL, sseURL, _, cleanup := runSSEServer(t)
    defer cleanup()
    tr := NewSSETransport(SSEConfig{PostURL: postURL, SSEURL:
        sseURL, BackoffOverride: 50 * time.Millisecond})
    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
    defer cancel()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()
    // give the SSE goroutine a moment to connect
    time.Sleep(100 * time.Millisecond)
    require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
        ID: "1", Method: "ping"}))
    resp, err := tr.Recv(ctx)
    require.NoError(t, err)
    assert.Equal(t, "1", resp.ID)
    assert.Equal(t, TransportSSE, tr.Type())
}

```

```

func TestSSETransport_ReconnectAfterStreamClose(t *testing.T) {
    postURL, sseURL, closeStream, cleanup := runSSEServer(t)
    defer cleanup()
    tr := NewSSETransport(SSEConfig{PostURL: postURL, SSEURL:
        sseURL, BackoffOverride: 50 * time.Millisecond})
    ctx, cancel := context.WithTimeout(context.Background(),
        10*time.Second)
    defer cancel()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()
    time.Sleep(100 * time.Millisecond)
    closeStream()
    // wait for reconnect
    deadline := time.Now().Add(3 * time.Second)
    for time.Now().Before(deadline) {
        if tr.Reconnects() >= 1 {
            break
        }
        time.Sleep(50 * time.Millisecond)
    }
    assert.GreaterOrEqual(t, tr.Reconnects(), int64(1))
    // after reconnect, send/recv must work
    time.Sleep(200 * time.Millisecond)
    require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
        ID: "2", Method: "ping"}))
    resp, err := tr.Recv(ctx)
    require.NoError(t, err)
    assert.Equal(t, "2", resp.ID)
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestSSETransport" ./
    internal/mcp/...

```

Expected: FAIL with undefined: NewSSETransport, SSEConfig.



Step 3: Write transport_sse.go

Create helix_code/internal/mcp/transport_sse.go:

```

package mcp

import (
    "bufio"
    "bytes"
    "context"

```

```

"encoding/json"
"fmt"
"io"
"net/http"
"strings"
"sync"
"sync/atomic"
"time"

"golang.org/x/oauth2"
)

type SSEConfig struct {
    PostURL      string
    SSEURL       string
    TokenSource   oauth2.TokenSource
    OAuthEnabled bool
    Headers      map[string]string
    BackoffOverride time.Duration // 0 = use BackoffSchedule; >0
                = fixed delay (test only)
}

type sseTransport struct {
    cfg      SSEConfig
    client    *http.Client
    recvCh    chan *recvItem
    cancel    context.CancelFunc
    mu        sync.Mutex
    closed    bool
    reconnects atomic.Int64
}

func NewSSETransport(cfg SSEConfig) *sseTransport {
    return &sseTransport{
        cfg:      cfg,
        client: &http.Client{Timeout: 0}, // SSE needs no timeout
        recvCh: make(chan *recvItem, 16),
    }
}

func (t *sseTransport) Type() TransportType { return TransportSSE }
func (t *sseTransport) Reconnects() int64    { return t.reconnects.Load() }

func (t *sseTransport) Open(ctx context.Context) error {
    if t.cfg.PostURL == "" || t.cfg.SSEURL == "" {
        return fmt.Errorf("mcp sse: empty URL(s)")
    }
}

```

```

    }
    rctx, cancel := context.WithCancel(ctx)
    t.cancel = cancel
    go t.runSSELoop(rctx)
    return nil
}

```

```

func (t *sseTransport) runSSELoop(ctx context.Context) {
    bs := NewBackoffSchedule()
    first := true
    for {
        if ctx.Err() != nil {
            return
        }
        if !first {
            t.reconnects.Add(1)
            delay := t.delay(bs)
            select {
            case <-time.After(delay):
            case <-ctx.Done():
                return
            }
        }
        first = false
        err := t.streamOnce(ctx)
        if ctx.Err() != nil {
            return
        }
        if err != nil {
            // loop will reconnect
            continue
        }
        bs.Reset()
    }
}

```

```

func (t *sseTransport) delay(bs *BackoffSchedule) time.Duration {
    if t.cfg.BackoffOverride > 0 {
        return t.cfg.BackoffOverride
    }
    return bs.Next()
}

```

```

func (t *sseTransport) streamOnce(ctx context.Context) error {
    req, err := http.NewRequestWithContext(ctx, "GET",
        t.cfg.SSEURL, nil)
    if err != nil {
        return err
    }

```

```

}
req.Header.Set("Accept", "text/event-stream")
for k, v := range t.cfg.Headers {
    req.Header.Set(k, v)
}
if t.cfg.TokenSource != nil {
    tok, err := t.cfg.TokenSource.Token()
    if err != nil {
        return fmt.Errorf("%w: %v", ErrOAuthRequired, err)
    }
    req.Header.Set("Authorization", "Bearer "+tok.AccessToken)
}
resp, err := t.client.Do(req)
if err != nil {
    return err
}
defer resp.Body.Close()
if resp.StatusCode != 200 {
    return fmt.Errorf("%w: SSE GET status %d", ErrProtocol, resp.StatusCode)
}
rd := bufio.NewReaderSize(resp.Body, 1024*1024)
var data bytes.Buffer
for {
    line, err := rd.ReadBytes('\n')
    if err != nil {
        if err == io.EOF {
            return nil
        }
        return err
    }
    line = bytes.TrimRight(line, "\r\n")
    if len(line) == 0 {
        if data.Len() > 0 {
            t.dispatch(data.Bytes())
            data.Reset()
        }
        continue
    }
    if bytes.HasPrefix(line, []byte(":")) {
        continue
    }
    if bytes.HasPrefix(line, []byte("data:")) {
        payload := bytes.TrimPrefix(line, []byte("data:"))
        payload = bytes.TrimPrefix(payload, []byte(" "))
        data.Write(payload)
        data.WriteByte('\n')
    }
}

```

```

    }
}

func (t *sseTransport) dispatch(data []byte) {
    data = bytes.TrimRight(data, "\n")
    var m MCPMessage
    if err := json.Unmarshal(data, &m); err != nil {
        t.recvCh <- &recvItem{err: fmt.Errorf("%w: SSE parse: %v", ErrProtocol, err)}
        return
    }
    t.recvCh <- &recvItem{msg: &m}
}

func (t *sseTransport) Send(ctx context.Context, msg
    *MCPMessage) error {
    t.mu.Lock()
    if t.closed {
        t.mu.Unlock()
        return ErrTransportClosed
    }
    t.mu.Unlock()
    body, err := json.Marshal(msg)
    if err != nil {
        return err
    }
    req, err := http.NewRequestWithContext(ctx, "POST",
        t.cfg.PostURL, bytes.NewReader(body))
    if err != nil {
        return err
    }
    req.Header.Set("Content-Type", "application/json")
    for k, v := range t.cfg.Headers {
        req.Header.Set(k, v)
    }
    if t.cfg.TokenSource != nil {
        tok, err := t.cfg.TokenSource.Token()
        if err != nil {
            return fmt.Errorf("%w: %v", ErrOAuthRequired, err)
        }
        req.Header.Set("Authorization", "Bearer
            "+tok.AccessToken)
    }
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return err
    }
}

```

```

defer resp.Body.Close()
if resp.StatusCode == 401 && t.cfg.OAuthEnabled {
    return ErrOAuthRequired
}
if resp.StatusCode < 200 || resp.StatusCode >= 300 {
    body, _ := io.ReadAll(io.LimitReader(resp.Body, 8192))
    return fmt.Errorf("%w: SSE POST %d: %s", ErrProtocol,
        resp.StatusCode, strings.TrimSpace(string(body)))
}
return nil
}

func (t *sseTransport) Recv(ctx context.Context) (*MCPMessage,
    error) {
    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case item := <-t.recvCh:
        if item.err != nil {
            return nil, item.err
        }
        return item.msg, nil
    }
}

func (t *sseTransport) Close() error {
    t.mu.Lock()
    if t.closed {
        t.mu.Unlock()
        return nil
    }
    t.closed = true
    t.mu.Unlock()
    if t.cancel != nil {
        t.cancel()
    }
    return nil
}

```



Step 4: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -run "TestSSETransport" ./
    internal/mcp/...

```

Expected: PASS (2/2).



Step 5: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/transport_sse.go helix_code/
    internal/mcp/transport_sse_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T05): add SSE MCP transport with auto-reconnect
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
    <noreply@anthropic.com>
```

```
EOF
)"
```

Task 6: transport_ws.go via gorilla/websocket (TDD)

Files: - Create: helix_code/internal/mcp/transport_ws.go - Create:
helix_code/internal/mcp/transport_ws_test.go



Step 1: Write failing test

Create helix_code/internal/mcp/transport_ws_test.go:

```
package mcp

import (
    "context"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
    "time"

    "github.com/gorilla/websocket"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func runWSEchoServer(t *testing.T) (string, func()) {
    t.Helper()
```

```

upgrader := websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}
srv := httptest.NewServer(http.HandlerFunc(func(w
    http.ResponseWriter, r *http.Request) {
        c, err := upgrader.Upgrade(w, r, nil)
        if err != nil {
            return
        }
        defer c.Close()
        for {
            _, msg, err := c.ReadMessage()
            if err != nil {
                return
            }
            var req MCPMessage
            if err := json.Unmarshal(msg, &req); err != nil {
                return
            }
            resp := MCPMessage{JSONRPC: "2.0", ID: req.ID,
                Result: map[string]any{"ok": true}}
            b, _ := json.Marshal(&resp)
            c.WriteMessage(websocket.TextMessage, b)
        }
    })
}
cleanup := func() { srv.Close() }
url := "ws" + strings.TrimPrefix(srv.URL, "http")
return url, cleanup
}

func TestWSTransport_RoundTrip(t *testing.T) {
    url, cleanup := runWSEchoServer(t)
    defer cleanup()
    tr := NewWSTransport(WSConfig{URL: url})
    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
    defer cancel()
    require.NoError(t, tr.Open(ctx))
    defer tr.Close()
    require.NoError(t, tr.Send(ctx, &MCPMessage{JSONRPC: "2.0",
        ID: "1", Method: "ping"}))
    resp, err := tr.Recv(ctx)
    require.NoError(t, err)
    assert.Equal(t, "1", resp.ID)
    assert.Equal(t, TransportWS, tr.Type())
}

func TestWSTransport_CloseStopsRecv(t *testing.T) {

```

```

url, cleanup := runWSEchoServer(t)
defer cleanup()
tr := NewWSTransport(WConfig{URL: url})
ctx, cancel := context.WithTimeout(context.Background(),
    5*time.Second)
defer cancel()
require.NoError(t, tr.Open(ctx))
require.NoError(t, tr.Close())
_, err := tr.Recv(ctx)
assert.Error(t, err)
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestWSTransport" ./
    internal/mcp/...

```

Expected: FAIL with undefined: NewWSTransport, WConfig.



Step 3: Write transport_ws.go

Create helix_code/internal/mcp/transport_ws.go:

```

package mcp

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "sync"
    "time"

    "github.com/gorilla/websocket"
    "golang.org/x/oauth2"
)

type WConfig struct {
    URL            string
    TokenSource     oauth2.TokenSource
    OAuthEnabled   bool
    Headers        http.Header
    PingInterval   time.Duration // 0 = 25s default
    PongTimeout    time.Duration // 0 = 30s default
}

type wsTransport struct {
    cfg WConfig
}

```

```

    conn      *websocket.Conn
    recvCh    chan *recvItem
    cancel    context.CancelFunc
    mu        sync.Mutex
    closed    bool
}

func NewWSTransport(cfg WSConfig) *wsTransport {
    return &wsTransport{
        cfg:      cfg,
        recvCh:   make(chan *recvItem, 16),
    }
}

func (t *wsTransport) Type() TransportType { return TransportWS }

func (t *wsTransport) Open(ctx context.Context) error {
    if t.cfg.URL == "" {
        return fmt.Errorf("mcp ws: empty URL")
    }
    hdr := http.Header{}
    for k, v := range t.cfg.Headers {
        hdr[k] = v
    }
    if t.cfg.TokenSource != nil {
        tok, err := t.cfg.TokenSource.Token()
        if err != nil {
            return fmt.Errorf("%w: %v", ErrOAuthRequired, err)
        }
        hdr.Set("Authorization", "Bearer "+tok.AccessToken)
    }
    dialer := websocket.DefaultDialer
    c, resp, err := dialer.DialContext(ctx, t.cfg.URL, hdr)
    if err != nil {
        if resp != nil && resp.StatusCode == 401 &&
            t.cfg.OAuthEnabled {
            return ErrOAuthRequired
        }
        return fmt.Errorf("mcp ws: dial: %w", err)
    }
    t.conn = c
    pong := t.cfg.PongTimeout
    if pong == 0 {
        pong = 30 * time.Second
    }
    c.SetReadLimit(16 * 1024 * 1024)
    c.SetReadDeadline(time.Now().Add(pong))
    c.SetPongHandler(func(string) error {

```

```

        c.SetReadDeadline(time.Now().Add(pong))
        return nil
    })
    rctx, cancel := context.WithCancel(ctx)
    t.cancel = cancel
    go t.readLoop(rctx)
    go t.pingLoop(rctx)
    return nil
}

func (t *wsTransport) readLoop(ctx context.Context) {
    for {
        if ctx.Err() != nil {
            return
        }
        _, data, err := t.conn.ReadMessage()
        if err != nil {
            t.recvCh <- &recvItem{err: fmt.Errorf("mcp ws: read:
            %w", err)}
            return
        }
        var m MCPMessage
        if err := json.Unmarshal(data, &m); err != nil {
            t.recvCh <- &recvItem{err: fmt.Errorf("%w: ws parse:
            %v", ErrProtocol, err)}
            continue
        }
        t.recvCh <- &recvItem{msg: &m}
    }
}

func (t *wsTransport) pingLoop(ctx context.Context) {
    iv := t.cfg.PingInterval
    if iv == 0 {
        iv = 25 * time.Second
    }
    tk := time.NewTicker(iv)
    defer tk.Stop()
    for {
        select {
        case <-ctx.Done():
            return
        case <-tk.C:
            t.mu.Lock()
            if t.conn != nil && !t.closed {
                _ = t.conn.WriteControl(websocket.PingMessage,
                nil, time.Now().Add(5*time.Second))
            }
        }
    }
}

```

```

        t.mu.Unlock()
    }
}

func (t *wsTransport) Send(ctx context.Context, msg *MCPMessage)
    error {
    t.mu.Lock()
    defer t.mu.Unlock()
    if t.closed || t.conn == nil {
        return ErrTransportClosed
    }
    b, err := json.Marshal(msg)
    if err != nil {
        return fmt.Errorf("mcp ws: marshal: %w", err)
    }
    return t.conn.WriteMessage(websocket.TextMessage, b)
}

func (t *wsTransport) Recv(ctx context.Context) (*MCPMessage,
    error) {
    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case item, ok := <-t.recvCh:
        if !ok {
            return nil, ErrTransportClosed
        }
        if item.err != nil {
            return nil, item.err
        }
        return item.msg, nil
    }
}

func (t *wsTransport) Close() error {
    t.mu.Lock()
    if t.closed {
        t.mu.Unlock()
        return nil
    }
    t.closed = true
    conn := t.conn
    t.mu.Unlock()
    if t.cancel != nil {
        t.cancel()
    }
    if conn != nil {

```

```

        _ = conn.WriteControl(websocket.CloseMessage,
                               websocket.FormatCloseMessage(websocket.CloseNormalClosure,
                                                             ""),
                               time.Now().Add(time.Second))
        _ = conn.Close()
    }
    return nil
}

```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestWSTransport" ./
internal/mcp/...
```

Expected: PASS (2/2).



Step 5: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/transport_ws.go helix_code/
    internal/mcp/transport_ws_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T06): add WebSocket MCP transport via gorilla/
websocket"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

```
EOF
)"
```

Task 7: oauth.go — RFC 8414 discovery + PKCE + token cache (TDD)

Files: - Create: helix_code/internal/mcp/oauth.go - Create: helix_code/internal/mcp/oauth_test.go



Step 1: Write failing test

Create `helix_code/internal/mcp/oauth_test.go`:

```
package mcp

import (
    "context"
    "crypto/sha256"
    "encoding/base64"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "net/url"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestPKCE_VerifierAndChallenge(t *testing.T) {
    v, c, err := generatePKCE()
    require.NoError(t, err)
    assert.GreaterOrEqual(t, len(v), 43)
    assert.LessOrEqual(t, len(v), 128)
    // challenge = base64url(sha256(verifier)), no padding
    sum := sha256.Sum256([]byte(v))
    want := base64.RawURLEncoding.EncodeToString(sum[:])
    assert.Equal(t, want, c)
}

func TestOAuth_DiscoverASMetadata(t *testing.T) {
    mux := http.NewServeMux()
    mux.HandleFunc("/.well-known/oauth-authorization-server",
        func(w http.ResponseWriter, r *http.Request) {
            _ = json.NewEncoder(w).Encode(&ASMetadata{
                Issuer: "https://example.com",
                AuthorizationEndpoint: "https://example.com/authz",
                TokenEndpoint: "https://example.com/token",
            })
        })
    srv := httptest.NewServer(mux)
    defer srv.Close()
    md, err := DiscoverAS(context.Background(), srv.URL)
    require.NoError(t, err)
}
```

```

    assert.Equal(t, "https://example.com/authz",
        md.AuthorizationEndpoint)
}

func TestOAuth_ExchangeCode(t *testing.T) {
    mux := http.NewServeMux()
    mux.HandleFunc("/token", func(w http.ResponseWriter, r
        *http.Request) {
        require.NoError(t, r.ParseForm())
        assert.Equal(t, "authorization_code",
            r.Form.Get("grant_type"))
        assert.Equal(t, "code-xyz", r.Form.Get("code"))
        assert.NotEmpty(t, r.Form.Get("code_verifier"))
        _ = json.NewEncoder(w).Encode(map[string]any{
            "access_token": "tok-1",
            "token_type": "Bearer",
            "refresh_token": "ref-1",
            "expires_in": 3600,
        })
    })
    srv := httptest.NewServer(mux)
    defer srv.Close()
    tok, err := ExchangeCode(context.Background(), srv.URL+"/
        token", "code-xyz", "verifier-12345-abcdefghij-
        klmnopqrst-uvwxyz", "client-id", "")
    require.NoError(t, err)
    assert.Equal(t, "tok-1", tok.AccessToken)
}

func TestTokenCache_PersistAndLoad(t *testing.T) {
    dir := t.TempDir()
    tc := &TokenCache{Dir: dir}
    tok := &SavedToken{AccessToken: "tok-1", RefreshToken:
        "ref-1", TokenType: "Bearer"}
    require.NoError(t, tc.Save("server-a", tok))
    // file exists with mode 0600
    path := filepath.Join(dir, "server-a.json")
    st, err := os.Stat(path)
    require.NoError(t, err)
    assert.Equal(t, os.FileMode(0600), st.Mode().Perm())
    got, err := tc.Load("server-a")
    require.NoError(t, err)
    assert.Equal(t, "tok-1", got.AccessToken)
}

func TestTokenCache_RefuseLooseMode(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "server-b.json")

```

```

require.NoError(t, os.WriteFile(path,
    []byte(`{"access_token":"x"}`), 0644))
tc := &TokenCache{Dir: dir}
_, err := tc.Load("server-b")
require.Error(t, err)
}

func TestAuthorizationURL_BuildsExpected(t *testing.T) {
    u := BuildAuthorizationURL(AuthRequest{
        AuthorizationEndpoint: "https://example.com/authz",
        ClientID:               "cid",
        RedirectURI:            "http://127.0.0.1:9000/callback",
        Scope:                  "tools",
        State:                  "st-1",
        CodeChallenge:          "ch-1",
    })
    parsed, err := url.Parse(u)
    require.NoError(t, err)
    q := parsed.Query()
    assert.Equal(t, "code", q.Get("response_type"))
    assert.Equal(t, "cid", q.Get("client_id"))
    assert.Equal(t, "S256", q.Get("code_challenge_method"))
    assert.Equal(t, "ch-1", q.Get("code_challenge"))
    assert.Equal(t, "st-1", q.Get("state"))
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestPKCE|TestOAuth|
    TestTokenCache|TestAuthorizationURL" ./internal/mcp/...

```

Expected: FAIL with undefined: generatePKCE, DiscoverAS, ExchangeCode, TokenCache, etc.



Step 3: Write oauth.go

Create helix_code/internal/mcp/oauth.go:

```

package mcp

import (
    "context"
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/json"
    "fmt"
    "io"
    "net/http"

```

```

    "net/url"
    "os"
    "path/filepath"
    "strconv"
    "strings"
    "time"
)

// ASMetadata is the subset of RFC 8414 fields we use.
type ASMetadata struct {
    Issuer                string `json:"issuer"`
    AuthorizationEndpoint string `json:"authorization_endpoint"`
    TokenEndpoint         string `json:"token_endpoint"`
    RegistrationEndpoint string
    `json:"registration_endpoint,omitempty"`
    ScopesSupported       []string
    `json:"scopes_supported,omitempty"`
    ResponseTypesSupported []string
    `json:"response_types_supported,omitempty"`
}

// SavedToken is the persisted on-disk format for an OAuth token.
type SavedToken struct {
    AccessToken string `json:"access_token"`
    TokenType   string `json:"token_type"`
    RefreshToken string `json:"refresh_token,omitempty"`
    Expiry       time.Time `json:"expiry,omitempty"`
}

// AuthRequest captures the inputs for building an authorization
// URL.
type AuthRequest struct {
    AuthorizationEndpoint string
    ClientID              string
    RedirectURI           string
    Scope                 string
    State                 string
    CodeChallenge         string
}

// generatePKCE returns (verifier, challenge). Verifier is 64
// base64url
// characters of cryptographic randomness; challenge is the S256
// derivation.
func generatePKCE() (string, string, error) {
    raw := make([]byte, 48)
    if _, err := rand.Read(raw); err != nil {
        return "", "", err
    }
    return "", "", err
}

```

```

    }
    verifier := base64.RawURLEncoding.EncodeToString(raw)
    sum := sha256.Sum256([]byte(verifier))
    challenge := base64.RawURLEncoding.EncodeToString(sum[:])
    return verifier, challenge, nil
}

// randState returns a 32-byte base64url state parameter.
func randState() (string, error) {
    raw := make([]byte, 24)
    if _, err := rand.Read(raw); err != nil {
        return "", err
    }
    return base64.RawURLEncoding.EncodeToString(raw), nil
}

// DiscoverAS fetches /.well-known/oauth-authorization-server
// (RFC 8414).
func DiscoverAS(ctx context.Context, baseURL string)
    (*ASMetadata, error) {
    u := strings.TrimRight(baseURL, "/") + "/.well-known/oauth-
        authorization-server"
    req, err := http.NewRequestWithContext(ctx, "GET", u, nil)
    if err != nil {
        return nil, err
    }
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode != 200 {
        return nil, fmt.Errorf("mcp oauth: AS metadata status
            %d", resp.StatusCode)
    }
    body, err := io.ReadAll(io.LimitReader(resp.Body, 1<<20))
    if err != nil {
        return nil, err
    }
    var md ASMetadata
    if err := json.Unmarshal(body, &md); err != nil {
        return nil, fmt.Errorf("mcp oauth: parse AS metadata:
            %w", err)
    }
    return &md, nil
}

```

```
// BuildAuthorizationURL composes the user-facing authorization URL.
```

```
func BuildAuthorizationURL(r AuthRequest) string {  
    q := url.Values{}  
    q.Set("response_type", "code")  
    q.Set("client_id", r.ClientID)  
    q.Set("redirect_uri", r.RedirectURI)  
    if r.Scope != "" {  
        q.Set("scope", r.Scope)  
    }  
    q.Set("state", r.State)  
    q.Set("code_challenge", r.CodeChallenge)  
    q.Set("code_challenge_method", "S256")  
    sep := "?"  
    if strings.Contains(r.AuthorizationEndpoint, "?") {  
        sep = "&"  
    }  
    return r.AuthorizationEndpoint + sep + q.Encode()  
}
```

```
// ExchangeCode performs the RFC 6749 authorization-code grant with PKCE.
```

```
func ExchangeCode(ctx context.Context, tokenEndpoint, code, verifier, clientID, redirectURI string) (*SavedToken, error) {  
    form := url.Values{}  
    form.Set("grant_type", "authorization_code")  
    form.Set("code", code)  
    form.Set("code_verifier", verifier)  
    form.Set("client_id", clientID)  
    if redirectURI != "" {  
        form.Set("redirect_uri", redirectURI)  
    }  
    req, err := http.NewRequestWithContext(ctx, "POST", tokenEndpoint, strings.NewReader(form.Encode()))  
    if err != nil {  
        return nil, err  
    }  
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")  
    resp, err := http.DefaultClient.Do(req)  
    if err != nil {  
        return nil, err  
    }  
    defer resp.Body.Close()  
    body, _ := io.ReadAll(io.LimitReader(resp.Body, 1<<20))  
    if resp.StatusCode < 200 || resp.StatusCode >= 300 {
```

```

        return nil, fmt.Errorf("mcp oauth: token exchange status
        %d: %s", resp.StatusCode, string(body))
    }
    var raw struct {
        AccessToken string `json:"access_token"`
        TokenType    string `json:"token_type"`
        RefreshToken string `json:"refresh_token"`
        ExpiresIn    int    `json:"expires_in"`
    }
    if err := json.Unmarshal(body, &raw); err != nil {
        return nil, fmt.Errorf("mcp oauth: parse token: %w", err)
    }
    tok := &SavedToken{
        AccessToken: raw.AccessToken,
        TokenType:   raw.TokenType,
        RefreshToken: raw.RefreshToken,
    }
    if raw.ExpiresIn > 0 {
        tok.Expiry = time.Now().Add(time.Duration(raw.ExpiresIn)
            * time.Second)
    }
    return tok, nil
}

// TokenCache persists OAuth tokens at <Dir>/<server>.json with
// mode 0600.
type TokenCache struct {
    Dir string
}

func (c *TokenCache) path(server string) string {
    return filepath.Join(c.Dir, server+".json")
}

func (c *TokenCache) Save(server string, tok *SavedToken) error {
    if err := os.MkdirAll(c.Dir, 0700); err != nil {
        return fmt.Errorf("mcp oauth cache: mkdir: %w", err)
    }
    data, err := json.MarshalIndent(tok, "", " ")
    if err != nil {
        return err
    }
    path := c.path(server)
    if err := os.WriteFile(path, data, 0600); err != nil {
        return err
    }
    return nil
}

```

```

func (c *TokenCache) Load(server string) (*SavedToken, error) {
    path := c.path(server)
    st, err := os.Stat(path)
    if err != nil {
        return nil, err
    }
    if st.Mode().Perm() != 0600 {
        return nil, fmt.Errorf("mcp oauth cache: refusing %s:
        mode is %v, want 0600", path, st.Mode().Perm())
    }
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, err
    }
    var tok SavedToken
    if err := json.Unmarshal(data, &tok); err != nil {
        return nil, err
    }
    return &tok, nil
}

// expiresInSecondsString formats a duration; helper kept for
// symmetry.
func expiresInSecondsString(d time.Duration) string {
    return strconv.Itoa(int(d.Seconds()))
}

```



Step 4: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -run "TestPKCE|TestOAuth|
    TestTokenCache|TestAuthorizationURL" ./internal/mcp/...

```

Expected: PASS (6/6).



Step 5: Anti-bluff smoke

```

cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
    "clean"

```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/oauth.go helix_code/internal/mcp/
    oauth_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T07): add OAuth 2.0 + RFC 8414 discovery + PKCE +
    token cache for MCP

Co-Authored-By: Claude Opus 4.7 (1M context)
    <noreply@anthropic.com>

EOF
)"
```

Task 8: lifecycle.go — Client state machine + handshake (TDD)

Files: - Create: helix_code/internal/mcp/lifecycle.go - Create: helix_code/internal/mcp/lifecycle_test.go - Create: helix_code/internal/mcp/fake_transport_test.go (test-only fake transport)



Step 1: Write fake_transport_test.go (test helper)

Create helix_code/internal/mcp/fake_transport_test.go:

```
package mcp

import (
    "context"
    "sync"
)

// fakeTransport is a programmable transport for unit-testing
// Client.
type fakeTransport struct {
    mu      sync.Mutex
    sent    []*MCPMessage
    recvCh  chan *recvItem
    openErr error
    t       TransportType
    closed  bool
}

func newFakeTransport() *fakeTransport {
    return &fakeTransport{
        recvCh: make(chan *recvItem, 32),
        t:      TransportType("fake"),
    }
}
```

```

}

func (f *fakeTransport) Type() TransportType { return f.t }
func (f *fakeTransport) Open(ctx context.Context) error {
    if f.openErr != nil {
        return f.openErr
    }
    return nil
}
func (f *fakeTransport) Send(ctx context.Context, m *MCPMessage)
    error {
    f.mu.Lock()
    defer f.mu.Unlock()
    if f.closed {
        return ErrTransportClosed
    }
    f.sent = append(f.sent, m)
    return nil
}
func (f *fakeTransport) Recv(ctx context.Context) (*MCPMessage,
    error) {
    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case item := <-f.recvCh:
        if item.err != nil {
            return nil, item.err
        }
        return item.msg, nil
    }
}
func (f *fakeTransport) Close() error {
    f.mu.Lock()
    f.closed = true
    f.mu.Unlock()
    return nil
}

// pushReply queues a synthetic server response.
func (f *fakeTransport) pushReply(m *MCPMessage) {
    f.recvCh <- &recvItem{msg: m}
}

// pushError queues a synthetic transport error.
func (f *fakeTransport) pushError(err error) {
    f.recvCh <- &recvItem{err: err}
}

```

```
// sentMessages returns a snapshot of all messages Send received.
func (f *fakeTransport) sentMessages() []*MCPMessage {
    f.mu.Lock()
    defer f.mu.Unlock()
    out := make([]*MCPMessage, len(f.sent))
    copy(out, f.sent)
    return out
}
```



Step 2: Write failing test

Create `helix_code/internal/mcp/lifecycle_test.go`:

```
package mcp

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestClient_HandshakeSuccess(t *testing.T) {
    ft := newFakeTransport()
    c := NewClient("srv-a", ft)
    ctx, cancel := context.WithTimeout(context.Background(),
        2*time.Second)
    defer cancel()

    go func() {
        // observe the initialize request, send synthetic reply
        time.Sleep(50 * time.Millisecond)
        sent := ft.sentMessages()
        var initID interface{}
        for _, m := range sent {
            if m.Method == "initialize" {
                initID = m.ID
                break
            }
        }
        require.NotNil(t, initID)
        ft.pushReply(&MCPMessage{JSONRPC: "2.0", ID: initID,
            Result: map[string]any{"capabilities":
                map[string]any{"tools": map[string]any{}}}})
        // then tools/list
        time.Sleep(50 * time.Millisecond)
    }
}
```

```

var toolsID interface{}
for _, m := range ft.sentMessages() {
    if m.Method == "tools/list" {
        toolsID = m.ID
        break
    }
}
require.NotNil(t, toolsID)
ft.pushReply(&MCPMessage{JSONRPC: "2.0", ID: toolsID,
Result: map[string]any{"tools":
[]map[string]any{{"name": "echo"}}}})
}()

require.NoError(t, c.Connect(ctx))
assert.Equal(t, StateReady, c.State())
tools := c.Tools()
require.Len(t, tools, 1)
assert.Equal(t, "echo", tools[0].Name)
}

func TestClient_CallToolReturnsResult(t *testing.T) {
    ft := newFakeTransport()
    c := NewClient("srv-a", ft)
    ctx, cancel := context.WithTimeout(context.Background(),
        2*time.Second)
    defer cancel()

    go func() {
        time.Sleep(50 * time.Millisecond)
        // reply to initialize, tools/list
        for _, m := range ft.sentMessages() {
            if m.Method == "initialize" {
                ft.pushReply(&MCPMessage{JSONRPC: "2.0", ID:
m.ID, Result: map[string]any{}})
            }
        }
        time.Sleep(50 * time.Millisecond)
        for _, m := range ft.sentMessages() {
            if m.Method == "tools/list" {
                ft.pushReply(&MCPMessage{JSONRPC: "2.0", ID:
m.ID, Result: map[string]any{"tools":
[]map[string]any{}}})
            }
        }
    }()
    require.NoError(t, c.Connect(ctx))

    go func() {

```

```

        time.Sleep(50 * time.Millisecond)
        for _, m := range ft.sentMessages() {
            if m.Method == "tools/call" {
                ft.pushReply(&MCPMessage{JSONRPC: "2.0", ID:
m.ID, Result: map[string]any{"content":
[]map[string]any{{"type": "text", "text": "hello"}}}})
            }
        }
    }()
    res, err := c.CallTool(ctx, "echo", map[string]any{"x": 1})
    require.NoError(t, err)
    assert.NotNil(t, res)
}

func TestClient_StateTransitions(t *testing.T) {
    ft := newFakeTransport()
    c := NewClient("srv-a", ft)
    assert.Equal(t, StateDisconnected, c.State())
    c.setState(StateConnecting)
    assert.Equal(t, StateConnecting, c.State())
    c.setState(StateReady)
    assert.Equal(t, StateReady, c.State())
}

```



Step 3: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestClient" ./internal/
mcp/...

```

Expected: FAIL with undefined: NewClient.



Step 4: Write lifecycle.go

Create helix_code/internal/mcp/lifecycle.go:

```

package mcp

import (
    "context"
    "errors"
    "fmt"
    "io"
    "strconv"
    "sync"
    "sync/atomic"
)

```

```

// ExternalTool is what Manager exposes to tools/registry: a tool
    living on
// some MCP server, identified by server name + tool name.
type ExternalTool struct {
    Server string
    Name    string
    Schema  map[string]any // jsonSchema for inputSchema
    Title   string
    Desc    string
}

// CallResult is what CallTool returns.
type CallResult struct {
    Content []map[string]any
    IsError bool
    Raw     interface{}
}

// Client is one connected MCP server.
type Client struct {
    name        string
    transport    Transport
    state        atomic.Int32
    mu           sync.Mutex
    tools        []ExternalTool
    pending      map[string]chan *MCPMessage
    nextID       atomic.Int64
    done         chan struct{}
    onEvent      func(Event)
    pendCap      int
}

// NewClient builds a Client around a Transport. transport.Open
    is called on Connect.
func NewClient(name string, transport Transport) *Client {
    c := &Client{
        name:        name,
        transport:    transport,
        pending:      make(map[string]chan *MCPMessage),
        done:         make(chan struct{}),
        pendCap:      1024,
    }
    c.state.Store(int32(StateDisconnected))
    return c
}

// Name returns the configured server name.
func (c *Client) Name() string { return c.name }

```

```

// Tools returns the cached tool list (post-handshake).
func (c *Client) Tools() []ExternalTool {
    c.mu.Lock()
    defer c.mu.Unlock()
    out := make([]ExternalTool, len(c.tools))
    copy(out, c.tools)
    return out
}

// State returns the current state.
func (c *Client) State() ClientState { return
    ClientState(c.state.Load()) }

func (c *Client) setState(s ClientState) {
    c.state.Store(int32(s))
    if c.onEvent != nil {
        c.onEvent(Event{Server: c.name, State: s})
    }
}

// SetOnEvent installs an event callback.
func (c *Client) SetOnEvent(fn func(Event)) { c.onEvent = fn }

func (c *Client) nextRPCID() string {
    return "rpc-" + strconv.FormatInt(c.nextID.Add(1), 10)
}

// Connect opens the transport and runs the initialize/tools-list
// handshake.
func (c *Client) Connect(ctx context.Context) error {
    c.setState(StateConnecting)
    if err := c.transport.Open(ctx); err != nil {
        c.setState(StateDisconnected)
        return fmt.Errorf("mcp client %s: open: %w", c.name, err)
    }
    go c.recvLoop(ctx)
    c.setState(StateInitializing)
    if err := c.handshake(ctx); err != nil {
        c.setState(StateDisconnected)
        return err
    }
    c.setState(StateReady)
    return nil
}

func (c *Client) handshake(ctx context.Context) error {
    initID := c.nextRPCID()

```

```

initResp, err := c.callRaw(ctx, initID, "initialize",
    map[string]any{
        "protocolVersion": "2024-11-05",
        "capabilities":    map[string]any{},
        "clientInfo":      map[string]any{"name": "helixcode",
        "version": "1.0"},
    })
if err != nil {
    return fmt.Errorf("%w: initialize: %v", ErrInitFailed,
        err)
}
if initResp.Error != nil {
    return fmt.Errorf("%w: server rejected initialize: %v",
        ErrInitFailed, initResp.Error)
}

// notifications/initialized (no id)
if err := c.transport.Send(ctx, &MCPMessage{JSONRPC: "2.0",
    Method: "notifications/initialized"}); err != nil {
    return fmt.Errorf("%w: notifications/initialized: %v",
        ErrInitFailed, err)
}

// tools/list
listID := c.nextRPCID()
listResp, err := c.callRaw(ctx, listID, "tools/list",
    map[string]any{})
if err != nil {
    return fmt.Errorf("%w: tools/list: %v", ErrInitFailed,
        err)
}
c.mu.Lock()
c.tools = parseToolsListResult(listResp.Result)
c.mu.Unlock()
return nil
}

func parseToolsListResult(raw interface{}) []ExternalTool {
    out := []ExternalTool{}
    m, ok := raw.(map[string]any)
    if !ok {
        return out
    }
    tools, ok := m["tools"].([]interface{})
    if !ok {
        return out
    }
    for _, item := range tools {

```

```

    td, ok := item.(map[string]any)
    if !ok {
        continue
    }
    name, _ := td["name"].(string)
    if name == "" {
        continue
    }
    t := ExternalTool{Name: name}
    if v, ok := td["title"].(string); ok {
        t.Title = v
    }
    if v, ok := td["description"].(string); ok {
        t.Desc = v
    }
    if v, ok := td["inputSchema"].(map[string]any); ok {
        t.Schema = v
    }
    out = append(out, t)
}
return out
}

```

// CallTool invokes a tool on the server. State must be ready.

```

func (c *Client) CallTool(ctx context.Context, name string, args
    map[string]any) (*CallResult, error) {
    if c.State() != StateReady {
        return nil, ErrNotReady
    }
    id := c.nextRPCID()
    resp, err := c.callRaw(ctx, id, "tools/call", map[string]any{
        "name":      name,
        "arguments": args,
    })
    if err != nil {
        return nil, err
    }
    if resp.Error != nil {
        if resp.Error.Code == -32601 {
            return nil, ErrToolNotFound
        }
        return nil, fmt.Errorf("mcp call %s: %s", name,
            resp.Error.Message)
    }
    cr := &CallResult{Raw: resp.Result}
    if rm, ok := resp.Result.(map[string]any); ok {
        if v, ok := rm["isError"].(bool); ok {
            cr.IsError = v
        }
    }
}

```

```

    }
    if items, ok := rm["content"].([]interface{}); ok {
        for _, it := range items {
            if m, ok := it.(map[string]any); ok {
                cr.Content = append(cr.Content, m)
            }
        }
    }
}
return cr, nil
}

func (c *Client) callRaw(ctx context.Context, id, method string,
    params map[string]any) (*MCPMessage, error) {
    c.mu.Lock()
    if len(c.pending) >= c.pendCap {
        c.mu.Unlock()
        return nil, ErrTooManyPending
    }
    ch := make(chan *MCPMessage, 1)
    c.pending[id] = ch
    c.mu.Unlock()
    defer func() {
        c.mu.Lock()
        delete(c.pending, id)
        c.mu.Unlock()
    }()

    if err := c.transport.Send(ctx, &MCPMessage{
        JSONRPC: "2.0",
        ID:      id,
        Method:  method,
        Params:  params,
    }); err != nil {
        return nil, err
    }
    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case <-c.done:
        return nil, ErrTransportClosed
    case msg := <-ch:
        return msg, nil
    }
}

func (c *Client) recvLoop(ctx context.Context) {
    for {

```

```

msg, err := c.transport.Recv(ctx)
if err != nil {
    if errors.Is(err, io.EOF) || errors.Is(err,
ErrTransportClosed) || ctx.Err() != nil {
        return
    }
    // fail all pending
    c.mu.Lock()
    for id, ch := range c.pending {
        select {
            case ch <- &MCPMessage{Error: &MCPErr{Code:
-32000, Message: err.Error()}}, ID: id}:
            default:
        }
    }
    c.mu.Unlock()
    return
}
if msg == nil {
    continue
}
idStr := messageIDString(msg.ID)
if idStr == "" {
    // notification – ignore for now
    continue
}
c.mu.Lock()
ch, ok := c.pending[idStr]
c.mu.Unlock()
if ok {
    select {
        case ch <- msg:
        default:
    }
}
}
}

```

```

func messageIDString(id interface{}) string {
    switch v := id.(type) {
        case string:
            return v
        case float64:
            return strconv.FormatFloat(v, 'f', -1, 64)
        case int:
            return strconv.Itoa(v)
        case nil:
            return ""
    }
}

```

```

        default:
            return fmt.Sprintf("%v", v)
    }
}

// Close shuts the client down.
func (c *Client) Close() error {
    c.setState(StateClosed)
    select {
    case <-c.done:
    default:
        close(c.done)
    }
    return c.transport.Close()
}

```



Step 5: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestClient" ./internal/
mcp/...
```

Expected: PASS (3/3).



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 7: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/lifecycle.go helix_code/internal/
    mcp/lifecycle_test.go helix_code/internal/mcp/
    fake_transport_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T08): add MCP Client lifecycle + state machine +
    handshake"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

EOF
)"

Task 9: registry.go — Manager + tool merging (TDD)

Files: - Create: helix_code/internal/mcp/registry.go - Create: helix_code/internal/mcp/registry_test.go



Step 1: Write failing test

Create helix_code/internal/mcp/registry_test.go:

```
package mcp

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestManager_AddClientAndListTools(t *testing.T) {
    m := NewManager()
    c := NewClient("srv-a", newFakeTransport())
    m.addClient(c)
    // inject tools directly (skip handshake for unit test)
    c.tools = []ExternalTool{{Name: "echo"}, {Name: "time"}}
    c.state.Store(int32(StateReady))

    tools := m.Tools()
    require.Len(t, tools, 2)
    // must be prefixed with server name
    names := []string{tools[0].Server + ":" + tools[0].Name,
        tools[1].Server + ":" + tools[1].Name}
    assert.Contains(t, names, "srv-a:echo")
    assert.Contains(t, names, "srv-a:time")
}

func TestManager_CallToolRoutes(t *testing.T) {
    m := NewManager()
    ft := newFakeTransport()
    c := NewClient("srv-a", ft)
    c.tools = []ExternalTool{{Name: "echo"}}
    c.state.Store(int32(StateReady))
    m.addClient(c)

    ctx, cancel := context.WithTimeout(context.Background(),
        2*time.Second)
    defer cancel()
```

```

go func() {
    time.Sleep(20 * time.Millisecond)
    for _, msg := range ft.sentMessages() {
        if msg.Method == "tools/call" {
            ft.pushReply(&MCPMessage{JSONRPC: "2.0", ID:
            msg.ID, Result: map[string]any{"content":
            []map[string]any{"type": "text", "text": "ok"}}})
            return
        }
    }
}()
go c.recvLoop(ctx)

res, err := m.CallTool(ctx, "srv-a", "echo",
    map[string]any{"x": 1})
require.NoError(t, err)
assert.NotNil(t, res)
}

func TestManager_ServerNotFound(t *testing.T) {
    m := NewManager()
    _, err := m.CallTool(context.Background(), "missing", "x",
        nil)
    assert.ErrorIs(t, err, ErrServerNotFound)
}

func TestManager_StatusReportsAllClients(t *testing.T) {
    m := NewManager()
    a := NewClient("a", newFakeTransport())
    a.state.Store(int32(StateReady))
    a.tools = []ExternalTool{{Name: "x"}}
    b := NewClient("b", newFakeTransport())
    b.state.Store(int32(StateDisconnected))
    m.addClient(a)
    m.addClient(b)
    st := m.Status()
    require.Len(t, st, 2)
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestManager" ./internal/
mcp/...

```

Expected: FAIL with undefined: NewManager, addClient.



Step 3: Write registry.go

Create `helix_code/internal/mcp/registry.go`:

```
package mcp

import (
    "context"
    "fmt"
    "sort"
    "sync"
)

// ClientStatus is a snapshot for /mcp listing.
type ClientStatus struct {
    Name          string
    Transport     TransportType
    State         ClientState
    ToolCount     int
    LastError     string
}

// Manager aggregates Clients and exposes tools to the rest of
// HelixCode.
type Manager struct {
    mu          sync.RWMutex
    clients     map[string]*Client
    cfg         *Config
}

// NewManager returns an empty Manager.
func NewManager() *Manager {
    return &Manager{clients: map[string]*Client{}}
}

// SetConfig replaces the loaded config (used by Reload).
func (m *Manager) SetConfig(c *Config) {
    m.mu.Lock()
    m.cfg = c
    m.mu.Unlock()
}

// Config returns the active configuration (may be nil).
func (m *Manager) Config() *Config {
    m.mu.RLock()
    defer m.mu.RUnlock()
    return m.cfg
}
```

```

// addClient registers a Client (used by tests + Start/Reload).
func (m *Manager) addClient(c *Client) {
    m.mu.Lock()
    m.clients[c.Name()] = c
    m.mu.Unlock()
}

// Tools returns all tools across ready clients, server-prefixed.
func (m *Manager) Tools() []ExternalTool {
    m.mu.RLock()
    defer m.mu.RUnlock()
    out := []ExternalTool{}
    for _, c := range m.clients {
        if c.State() != StateReady {
            continue
        }
        for _, t := range c.Tools() {
            t.Server = c.Name()
            out = append(out, t)
        }
    }
    sort.Slice(out, func(i, j int) bool {
        if out[i].Server != out[j].Server {
            return out[i].Server < out[j].Server
        }
        return out[i].Name < out[j].Name
    })
    return out
}

// CallTool routes to the named server's Client.
func (m *Manager) CallTool(ctx context.Context, server, tool
    string, args map[string]any) (*CallResult, error) {
    m.mu.RLock()
    c, ok := m.clients[server]
    m.mu.RUnlock()
    if !ok {
        return nil, fmt.Errorf("%w: %s", ErrServerNotFound,
            server)
    }
    return c.CallTool(ctx, tool, args)
}

// Status returns a snapshot for /mcp listing.
func (m *Manager) Status() []ClientStatus {
    m.mu.RLock()
    defer m.mu.RUnlock()
    out := []ClientStatus{}

```

```

    for _, c := range m.clients {
        out = append(out, ClientStatus{
            Name:      c.Name(),
            Transport: c.transport.Type(),
            State:      c.State(),
            ToolCount: len(c.Tools()),
        })
    }
    sort.Slice(out, func(i, j int) bool { return out[i].Name <
        out[j].Name })
    return out
}

// Start dials all alwaysLoad clients in parallel. cfg must be
// set.
func (m *Manager) Start(ctx context.Context) error {
    m.mu.RLock()
    cfg := m.cfg
    m.mu.RUnlock()
    if cfg == nil {
        return nil
    }
    var wg sync.WaitGroup
    for _, srv := range cfg.Servers {
        c, err := m.buildClient(srv)
        if err != nil {
            return err
        }
        m.addClient(c)
        if !srv.AlwaysLoad {
            continue
        }
        wg.Add(1)
        go func(c *Client) {
            defer wg.Done()
            _ = c.Connect(ctx)
        }(c)
    }
    wg.Wait()
    return nil
}

// buildClient constructs a Client from a ServerSpec via the
// transport factory.
func (m *Manager) buildClient(s ServerSpec) (*Client, error) {
    tr, err := buildTransport(s)
    if err != nil {
        return nil, fmt.Errorf("mcp: server %s: %w", s.Name, err)
    }

```

```

    }
    return NewClient(s.Name, tr), nil
}

// Test runs a one-shot connect → tools/list → close cycle.
func (m *Manager) Test(ctx context.Context, name string) error {
    m.mu.RLock()
    cfg := m.cfg
    m.mu.RUnlock()
    if cfg == nil {
        return fmt.Errorf("mcp: no config loaded")
    }
    for _, s := range cfg.Servers {
        if s.Name != name {
            continue
        }
        tr, err := buildTransport(s)
        if err != nil {
            return err
        }
        c := NewClient(name, tr)
        if err := c.Connect(ctx); err != nil {
            return err
        }
        _ = c.Close()
        return nil
    }
    return ErrServerNotFound
}

// Reload diffs config and reconciles clients.
func (m *Manager) Reload(ctx context.Context, newCfg *Config)
    error {
    m.mu.Lock()
    old := m.cfg
    m.cfg = newCfg
    m.mu.Unlock()

    want := map[string]ServerSpec{}
    for _, s := range newCfg.Servers {
        want[s.Name] = s
    }
    have := map[string]ServerSpec{}
    if old != nil {
        for _, s := range old.Servers {
            have[s.Name] = s
        }
    }
}

```

```

// removed
for name := range have {
    if _, ok := want[name]; ok {
        continue
    }
    m.mu.Lock()
    c, ok := m.clients[name]
    delete(m.clients, name)
    m.mu.Unlock()
    if ok {
        _ = c.Close()
    }
}
// added or changed
for name, spec := range want {
    old, exists := have[name]
    if exists && specEqual(old, spec) {
        continue
    }
    if exists {
        m.mu.Lock()
        if c, ok := m.clients[name]; ok {
            _ = c.Close()
        }
        m.mu.Unlock()
    }
    c, err := m.buildClient(spec)
    if err != nil {
        return err
    }
    m.addClient(c)
    if spec.AlwaysLoad {
        go func(c *Client) { _ = c.Connect(ctx) }(c)
    }
}
return nil
}

// specEqual is a shallow equality for ServerSpec – used to skip
// unchanged servers.
func specEqual(a, b ServerSpec) bool {
    if a.Name != b.Name || a.Transport != b.Transport || a.URL !=
        b.URL || a.AlwaysLoad != b.AlwaysLoad {
        return false
    }
    if len(a.Command) != len(b.Command) {
        return false
    }

```

```

    }
    for i := range a.Command {
        if a.Command[i] != b.Command[i] {
            return false
        }
    }
    return true
}

```

```

// Close shuts every client down.
func (m *Manager) Close() error {
    m.mu.Lock()
    clients := make([]*Client, 0, len(m.clients))
    for _, c := range m.clients {
        clients = append(clients, c)
    }
    m.clients = map[string]*Client{}
    m.mu.Unlock()
    var wg sync.WaitGroup
    for _, c := range clients {
        wg.Add(1)
        go func(c *Client) {
            defer wg.Done()
            _ = c.Close()
        }(c)
    }
    wg.Wait()
    return nil
}

```

// buildTransport is a factory that maps a ServerSpec to a Transport.

```

func buildTransport(s ServerSpec) (Transport, error) {
    switch s.Transport {
    case TransportStdio:
        return NewStdioTransport(StdioConfig{
            Command: s.Command,
            Env:      s.Env,
            Cwd:      s.Cwd,
        }), nil
    case TransportHTTP:
        return NewHTTPTransport(HTTPConfig{
            URL:          s.URL,
            OAuthEnabled: s.OAuth.Enabled,
        }), nil
    case TransportSSE:
        return NewSSETransport(SSEConfig{
            PostURL:      s.URL,

```

```

        SSEURL:      s.SSEURL,
        OAuthEnabled: s.OAuth.Enabled,
    )), nil
case TransportWS:
    return NewWSTransport(WConfig{
        URL:      s.URL,
        OAuthEnabled: s.OAuth.Enabled,
    )), nil
default:
    return nil, fmt.Errorf("mcp: unsupported transport %q",
        s.Transport)
}
}

```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestManager" ./internal/
mcp/...
```

Expected: PASS (4/4).



Step 5: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/registry.go helix_code/internal/
    mcp/registry_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T09): add MCP Manager registry + tool merging +
    reload"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

```
EOF
)"
```

Task 10: config.go — YAML loader/saver, project + user precedence (TDD)

Files: - Create: helix_code/internal/mcp/config.go - Create: helix_code/internal/mcp/config_test.go



Step 1: Write failing test

Create helix_code/internal/mcp/config_test.go:

```
package mcp

import (
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestConfig_LoadFromYAML(t *testing.T) {
    yaml := []byte(`
servers:
  - name: brave
    transport: stdio
    command: ["npx", "@modelcontextprotocol/server-brave-search"]
    env:
      BRAVE_API_KEY: ${BRAVE_API_KEY}
    alwaysLoad: true
  - name: cloudflare
    transport: sse
    url: https://example.com/post
    sseURL: https://example.com/sse
    oauth:
      enabled: true
`)

    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "mcp.yml"), yaml, 0644))
    t.Setenv("BRAVE_API_KEY", "k-1")
    cfg, err := LoadConfig(filepath.Join(dir, "mcp.yml"))
    require.NoError(t, err)
    require.Len(t, cfg.Servers, 2)
    assert.Equal(t, "brave", cfg.Servers[0].Name)
    assert.Equal(t, TransportStdio, cfg.Servers[0].Transport)
    assert.Equal(t, "k-1", cfg.Servers[0].Env["BRAVE_API_KEY"])
```

```

    assert.True(t, cfg.Servers[0].AlwaysLoad)
    assert.Equal(t, TransportSSE, cfg.Servers[1].Transport)
    assert.True(t, cfg.Servers[1].OAuth.Enabled)
}

func TestConfig_ProjectOverridesUser(t *testing.T) {
    user := []byte(`
servers:
  - name: brave
    transport: stdio
    command: ["from-user"]
  - name: only-user
    transport: stdio
    command: ["x"]
`)
    project := []byte(`
servers:
  - name: brave
    transport: stdio
    command: ["from-project"]
  - name: only-project
    transport: stdio
    command: ["y"]
`)

    dir := t.TempDir()
    uPath := filepath.Join(dir, "user.yml")
    pPath := filepath.Join(dir, "project.yml")
    require.NoError(t, os.WriteFile(uPath, user, 0644))
    require.NoError(t, os.WriteFile(pPath, project, 0644))

    cfg, err := LoadMerged(uPath, pPath)
    require.NoError(t, err)
    specs := map[string]ServerSpec{}
    for _, s := range cfg.Servers {
        specs[s.Name] = s
    }
    require.Len(t, cfg.Servers, 3)
    assert.Equal(t, []string{"from-project"},
        specs["brave"].Command)
    assert.Equal(t, []string{"x"}, specs["only-user"].Command)
    assert.Equal(t, []string{"y"}, specs["only-project"].Command)
}

func TestConfig_ValidateRequiresTransport(t *testing.T) {
    yaml := []byte("servers:\n  - name: x\n")
    dir := t.TempDir()
    path := filepath.Join(dir, "mcp.yml")
    require.NoError(t, os.WriteFile(path, yaml, 0644))

```

```

_, err := LoadConfig(path)
require.Error(t, err)
}

func TestConfig_SaveRoundTrip(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "mcp.yml")
    cfg := &Config{
        Servers: []ServerSpec{
            {Name: "a", Transport: TransportStdio, Command:
                []string{"echo"}, AlwaysLoad: true},
        },
    }
    require.NoError(t, SaveConfig(path, cfg))
    got, err := LoadConfig(path)
    require.NoError(t, err)
    require.Len(t, got.Servers, 1)
    assert.Equal(t, "a", got.Servers[0].Name)
    assert.True(t, got.Servers[0].AlwaysLoad)
}

```



Step 2: Run test to verify it fails

```
cd HelixCode && go test -count=1 -run "TestConfig" ./internal/
mcp/...
```

Expected: FAIL with undefined: LoadConfig, LoadMerged, SaveConfig, Config, ServerSpec.



Step 3: Write config.go

Create helix_code/internal/mcp/config.go:

```

package mcp

import (
    "errors"
    "fmt"
    "os"
    "regexp"

    "gopkg.in/yaml.v3"
)

// Config is the top-level YAML schema for .helixcode/mcp.yml.
type Config struct {
    Servers []ServerSpec `yaml:"servers"`
}

```

```

// ServerSpec defines one MCP server.
type ServerSpec struct {
    Name          string          `yaml:"name"`
    Transport      TransportType `yaml:"transport"`
    Command        []string        `yaml:"command,omitempty" //
    stdio
    Env            map[string]string `yaml:"env,omitempty" //
    stdio
    Cwd            string          `yaml:"cwd,omitempty" //
    stdio
    URL            string          `yaml:"url,omitempty" //
    http / sse / ws (POST URL for sse)
    SSEURL         string          `yaml:"sseURL,omitempty" //
    sse only (event stream URL)
    OAuth          OAuthSpec       `yaml:"oauth,omitempty"`
    AlwaysLoad     bool           `yaml:"alwaysLoad,omitempty"`
}

// OAuthSpec describes the OAuth configuration for a server.
type OAuthSpec struct {
    Enabled        bool    `yaml:"enabled,omitempty"`
    ClientID       string `yaml:"clientID,omitempty"`
    Scope          string `yaml:"scope,omitempty"`
    IssuerURL      string `yaml:"issuerURL,omitempty"`
    AuthEndpoint   string `yaml:"authEndpoint,omitempty" //
    overrides discovery
    TokenEndpoint  string `yaml:"tokenEndpoint,omitempty" //
    overrides discovery
}

var envRe = regexp.MustCompile(`\$\{([A-Za-z_][A-Za-z0-9_]*)\}`)

func expandEnv(s string) string {
    return envRe.ReplaceAllStringFunc(s, func(m string) string {
        key := m[2 : len(m)-1]
        if v, ok := os.LookupEnv(key); ok {
            return v
        }
        return ""
    })
}

// LoadConfig reads and validates a single YAML file.
func LoadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("mcp config: read %s: %w", path, err)
    }
}

```

```

}
var cfg Config
if err := yaml.Unmarshal(data, &cfg); err != nil {
    return nil, fmt.Errorf("mcp config: parse %s: %w", path,
        err)
}
for i := range cfg.Servers {
    s := &cfg.Servers[i]
    s.URL = expandEnv(s.URL)
    s.SSEURL = expandEnv(s.SSEURL)
    s.Cwd = expandEnv(s.Cwd)
    for j, c := range s.Command {
        s.Command[j] = expandEnv(c)
    }
    for k, v := range s.Env {
        s.Env[k] = expandEnv(v)
    }
}
if err := cfg.Validate(); err != nil {
    return nil, err
}
return &cfg, nil
}

```

```

// LoadMerged loads userPath then projectPath, with project
// overriding by name.
// Either path may be empty or non-existent (returns just the
// other side).
func LoadMerged(userPath, projectPath string) (*Config, error) {
    merged := &Config{}
    addAll := func(c *Config) {
        for _, s := range c.Servers {
            merged.Servers = append(merged.Servers, s)
        }
    }
    if userPath != "" {
        if _, err := os.Stat(userPath); err == nil {
            c, err := LoadConfig(userPath)
            if err != nil {
                return nil, err
            }
            addAll(c)
        } else if !errors.Is(err, os.ErrNotExist) {
            return nil, err
        }
    }
    if projectPath != "" {
        if _, err := os.Stat(projectPath); err == nil {

```

```

        c, err := LoadConfig(projectPath)
        if err != nil {
            return nil, err
        }
        // project overrides: drop user entries with matching
names first
        byName := map[string]bool{}
        for _, s := range c.Servers {
            byName[s.Name] = true
        }
        filtered := merged.Servers[:0]
        for _, s := range merged.Servers {
            if !byName[s.Name] {
                filtered = append(filtered, s)
            }
        }
        merged.Servers = filtered
        addAll(c)
    } else if !errors.Is(err, os.ErrNotExist) {
        return nil, err
    }
}
return merged, nil
}

// SaveConfig writes the config back to YAML at path.
func SaveConfig(path string, cfg *Config) error {
    if err := cfg.Validate(); err != nil {
        return err
    }
    data, err := yaml.Marshal(cfg)
    if err != nil {
        return err
    }
    return os.WriteFile(path, data, 0644)
}

// Validate checks the config for required fields.
func (c *Config) Validate() error {
    seen := map[string]bool{}
    for i, s := range c.Servers {
        if s.Name == "" {
            return fmt.Errorf("mcp config: server %d: empty
name", i)
        }
        if seen[s.Name] {
            return fmt.Errorf("mcp config: duplicate server name
%q", s.Name)
        }
    }
}

```

```

    }
    seen[s.Name] = true
    if err := s.Transport.Validate(); err != nil {
        return fmt.Errorf("mcp config: server %s: %w",
            s.Name, err)
    }
    switch s.Transport {
    case TransportStdio:
        if len(s.Command) == 0 {
            return fmt.Errorf("mcp config: server %s: stdio
requires command", s.Name)
        }
    case TransportHTTP, TransportSSE, TransportWS:
        if s.URL == "" {
            return fmt.Errorf("mcp config: server %s: %s
requires url", s.Name, s.Transport)
        }
    }
}
return nil
}

```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestConfig" ./internal/
mcp/...
```

Expected: PASS (4/4).



Step 5: Run full unit test sweep

```
cd HelixCode && go test -count=1 ./internal/mcp/...
```

Expected: all unit tests PASS.



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/mcp/ && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 7: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/mcp/config.go helix_code/internal/
    mcp/config_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T10): add MCP YAML config loader/saver with project-
overrides-user merging
```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

EOF
)"

Task 11: cmd/cli/mcp_cmd.go + /mcp slash command (TDD)

Files: - Create: helix_code/cmd/cli/mcp_cmd.go - Create: helix_code/cmd/cli/mcp_cmd_test.go - Create: helix_code/internal/commands/mcp_command.go - Create: helix_code/internal/commands/mcp_command_test.go - Modify: helix_code/internal/commands/builtin/register.go — register /mcp



Step 1: Write failing test for cobra subcommands

Create helix_code/cmd/cli/mcp_cmd_test.go:

```
package main

import (
    "bytes"
    "context"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/mcp"
)

func TestMCPAdd_StdioWritesYAML(t *testing.T) {
    dir := t.TempDir()
    cfgPath := filepath.Join(dir, "mcp.yml")
    cmd := newMCPCommand(MCPCommandDeps{ConfigPath: cfgPath})
    cmd.SetArgs([]string{"add", "echo", "--transport=stdio", "--command", "echo", "--command", "hello"})
    require.NoError(t, cmd.Execute())
    cfg, err := mcp.LoadConfig(cfgPath)
```

```

    require.NoError(t, err)
    require.Len(t, cfg.Servers, 1)
    assert.Equal(t, mcp.TransportStdio, cfg.Servers[0].Transport)
    assert.Equal(t, []string{"echo", "hello"},
        cfg.Servers[0].Command)
}

```

```

func TestMCPRemove_DropsEntry(t *testing.T) {
    dir := t.TempDir()
    cfgPath := filepath.Join(dir, "mcp.yml")
    require.NoError(t, mcp.SaveConfig(cfgPath, &mcp.Config{
        Servers: []mcp.ServerSpec{
            {Name: "a", Transport: mcp.TransportStdio, Command:
                []string{"x"}},
            {Name: "b", Transport: mcp.TransportStdio, Command:
                []string{"y"}},
        },
    })))
    cmd := newMCPCCommand(MCPCCommandDeps{ConfigPath: cfgPath})
    cmd.SetArgs([]string{"remove", "a"})
    require.NoError(t, cmd.Execute())
    cfg, err := mcp.LoadConfig(cfgPath)
    require.NoError(t, err)
    require.Len(t, cfg.Servers, 1)
    assert.Equal(t, "b", cfg.Servers[0].Name)
}

```

```

func TestMCPList_PrintsTable(t *testing.T) {
    dir := t.TempDir()
    cfgPath := filepath.Join(dir, "mcp.yml")
    require.NoError(t, mcp.SaveConfig(cfgPath, &mcp.Config{
        Servers: []mcp.ServerSpec{{Name: "a", Transport:
            mcp.TransportStdio, Command: []string{"x"}}},
    })))
    cmd := newMCPCCommand(MCPCCommandDeps{ConfigPath: cfgPath})
    var buf bytes.Buffer
    cmd.SetOut(&buf)
    cmd.SetArgs([]string{"list"})
    require.NoError(t, cmd.Execute())
    assert.Contains(t, buf.String(), "a")
    assert.Contains(t, buf.String(), "stdio")
}

```

```

func TestMCPTest_InvokesManagerTest(t *testing.T) {
    dir := t.TempDir()
    cfgPath := filepath.Join(dir, "mcp.yml")
    require.NoError(t, mcp.SaveConfig(cfgPath, &mcp.Config{

```

```

        Servers: []mcp.ServerSpec{{Name: "a", Transport:
            mcp.TransportStdio, Command: []string{"true"}}},
    )))
    called := false
    cmd := newMCPCmd(MCPCmdDeps{
        ConfigPath: cfgPath,
        TestServer: func(ctx context.Context, name string) error
        {
            called = true
            assert.Equal(t, "a", name)
            return nil
        },
    })
    cmd.SetArgs([]string{"test", "a"})
    require.NoError(t, cmd.Execute())
    assert.True(t, called)
}

func TestMain(m *testing.M) {
    // guard against accidental writes outside tempdir
    os.Exit(m.Run())
}

```



Step 2: Run test to verify it fails

```
cd HelixCode && go test -count=1 -run "TestMCP" ./cmd/cli/...
```

Expected: FAIL with undefined: newMCPCmd, MCPCmdDeps.



Step 3: Write cmd/cli/mcp_cmd.go

Create helix_code/cmd/cli/mcp_cmd.go:

```

package main

import (
    "context"
    "fmt"
    "os"
    "strings"
    "text/tabwriter"
    "time"

    "github.com/spf13/cobra"

    "dev.helix.code/internal/mcp"
)

```

```

// MCPCommandDeps wires test seams.
type MCPCommandDeps struct {
    ConfigPath string //
        path to mcp.yml
    TestServer func(ctx context.Context, name string) error //
        override for unit tests
    Auth        func(ctx context.Context, name string) error
    Logs        func(name string) ([]byte, error)
    List        func() ([]mcp.ClientStatus, error)
}

func newMCPCommand(deps MCPCommandDeps) *cobra.Command {
    root := &cobra.Command{
        Use:    "mcp",
        Short: "Manage MCP server connections",
    }
    root.AddCommand(newMCPAdd(deps))
    root.AddCommand(newMCPRemove(deps))
    root.AddCommand(newMCPList(deps))
    root.AddCommand(newMCPTest(deps))
    root.AddCommand(newMCPAuth(deps))
    root.AddCommand(newMCPLogs(deps))
    return root
}

func loadOrEmpty(path string) (*mcp.Config, error) {
    if _, err := os.Stat(path); err != nil && os.IsNotExist(err) {
        return &mcp.Config{}, nil
    }
    return mcp.LoadConfig(path)
}

func newMCPAdd(deps MCPCommandDeps) *cobra.Command {
    var transport string
    var command []string
    var url string
    var sseURL string
    var oauth bool
    cmd := &cobra.Command{
        Use:    "add <name>",
        Short: "Add an MCP server",
        Args:  cobra.ExactArgs(1),
        RunE: func(cmd *cobra.Command, args []string) error {
            cfg, err := loadOrEmpty(deps.ConfigPath)
            if err != nil {
                return err
            }
        }
    }
}

```

```

        spec := mcp.ServerSpec{
            Name:      args[0],
            Transport: mcp.TransportType(transport),
            Command:   command,
            URL:       url,
            SSEURL:    sseURL,
            OAuth:     mcp.OAuthSpec{Enabled: oauth},
        }
        cfg.Servers = append(cfg.Servers, spec)
        if err := mcp.SaveConfig(deps.ConfigPath, cfg); err != nil {
            return err
        }
        fmt.Fprintf(cmd.OutOrStdout(), "added %s (%s)\n",
            spec.Name, spec.Transport)
        return nil
    },
}

cmd.Flags().StringVar(&transport, "transport", "stdio",
    "transport: stdio|http|sse|ws")
cmd.Flags().StringSliceVar(&command, "command", nil,
    "command argv (stdio); repeat the flag")
cmd.Flags().StringVar(&url, "url", "", "URL (http/sse/ws;
    sse: POST URL)")
cmd.Flags().StringVar(&sseURL, "sse-url", "", "SSE event-
    stream URL")
cmd.Flags().BoolVar(&oauth, "oauth", false,
    "enable OAuth for this server")
return cmd
}

func newMCPRemove(deps MCPCommandDeps) *cobra.Command {
    return &cobra.Command{
        Use:      "remove <name>",
        Short:    "Remove an MCP server",
        Args:    cobra.ExactArgs(1),
        RunE:    func(cmd *cobra.Command, args []string) error {
            cfg, err := loadOrEmpty(deps.ConfigPath)
            if err != nil {
                return err
            }
            out := cfg.Servers[:0]
            found := false
            for _, s := range cfg.Servers {
                if s.Name == args[0] {
                    found = true
                    continue
                }
            }

```

```

        out = append(out, s)
    }
    if !found {
        return fmt.Errorf("mcp: server %q not found",
args[0])
    }
    cfg.Servers = out
    if err := mcp.SaveConfig(deps.ConfigPath, cfg); err !=
nil {
        return err
    }
    fmt.Fprintf(cmd.OutOrStdout(), "removed %s\n",
args[0])
    return nil
},
}
}

```

```

func newMCPList(deps MCPCommandDeps) *cobra.Command {
    return &cobra.Command{
        Use: "list",
        Short: "List configured MCP servers",
        RunE: func(cmd *cobra.Command, args []string) error {
            cfg, err := loadOrEmpty(deps.ConfigPath)
            if err != nil {
                return err
            }
            tw := tabwriter.NewWriter(cmd.OutOrStdout(), 0, 0,
2, ' ', 0)
            fmt.Fprintln(tw, "NAME\tTRANSPORT\tALWAYS-
LOAD\tTARGET")
            for _, s := range cfg.Servers {
                target := s.URL
                if s.Transport == mcp.TransportStdio {
                    target = strings.Join(s.Command, " ")
                }
                fmt.Fprintf(tw, "%s\t%s\t\t\t%s\n", s.Name,
s.Transport, s.AlwaysLoad, target)
            }
            return tw.Flush()
        },
    }
}

```

```

func newMCPTest(deps MCPCommandDeps) *cobra.Command {
    return &cobra.Command{
        Use: "test <name>",
        Short: "Probe a server (connect → tools/list → close)",
    }
}

```

```

Args: cobra.ExactArgs(1),
RunE: func(cmd *cobra.Command, args []string) error {
    ctx, cancel := context.WithTimeout(cmd.Context(),
30*time.Second)
    defer cancel()
    if deps.TestServer != nil {
        if err := deps.TestServer(ctx, args[0]); err !=
nil {
            return err
        }
        fmt.Fprintf(cmd.OutOrStdout(), "ready\n")
        return nil
    }
    cfg, err := loadOrEmpty(deps.ConfigPath)
    if err != nil {
        return err
    }
    m := mcp.NewManager()
    m.SetConfig(cfg)
    if err := m.Test(ctx, args[0]); err != nil {
        return err
    }
    fmt.Fprintf(cmd.OutOrStdout(), "ready\n")
    return nil
},
}

```

```

func newMCPAuth(deps MCPCommandDeps) *cobra.Command {
    return &cobra.Command{
        Use: "auth <name>",
        Short: "Run OAuth flow for a server",
        Args: cobra.ExactArgs(1),
        RunE: func(cmd *cobra.Command, args []string) error {
            ctx := cmd.Context()
            if deps.Auth != nil {
                return deps.Auth(ctx, args[0])
            }
            return runOAuthInteractive(ctx, deps.ConfigPath,
args[0], cmd.OutOrStdout())
        },
    }
}

```

```

func newMCPLogs(deps MCPCommandDeps) *cobra.Command {
    return &cobra.Command{
        Use: "logs <name>",

```

```

Short: "Show recent stderr + lifecycle events for a
server",
Args: cobra.ExactArgs(1),
RunE: func(cmd *cobra.Command, args []string) error {
    if deps.Logs != nil {
        b, err := deps.Logs(args[0])
        if err != nil {
            return err
        }
        cmd.OutOrStdout().Write(b)
        return nil
    }
    fmt.Fprintln(cmd.OutOrStdout(),
"(logs available only on a running helixcode instance
via /mcp logs)")
    return nil
},
}
}

```



Step 4: Add OAuth interactive helper

Append to helix_code/cmd/cli/mcp_cmd.go:

```

// runOAuthInteractive performs the PKCE flow against the named
// server.
// It prints the authorization URL, opens it in the user's
// browser, runs a
// one-shot loopback http.Server on 127.0.0.1:0/callback, and
// persists the
// resulting token to ~/.config/helixcode/mcp/tokens/<name>.json.
func runOAuthInteractive(ctx context.Context, configPath, name
    string, out interface {
        Write([]byte) (int, error)
    }) error {
    cfg, err := loadOrEmpty(configPath)
    if err != nil {
        return err
    }
    var spec *mcp.ServerSpec
    for i := range cfg.Servers {
        if cfg.Servers[i].Name == name {
            spec = &cfg.Servers[i]
            break
        }
    }
    if spec == nil {
        return fmt.Errorf("mcp: server %q not found", name)
    }
}

```

```

}
if !spec.OAuth.Enabled {
    return fmt.Errorf("mcp: server %q has
        oauth.enabled=false", name)
}
// Discovery (or use overrides)
authEP := spec.OAuth.AuthEndpoint
tokEP := spec.OAuth.TokenEndpoint
if authEP == "" || tokEP == "" {
    base := spec.OAuth.IssuerURL
    if base == "" {
        base = spec.URL
    }
    md, err := mcp.DiscoverAS(ctx, base)
    if err != nil {
        return err
    }
    if authEP == "" {
        authEP = md.AuthorizationEndpoint
    }
    if tokEP == "" {
        tokEP = md.TokenEndpoint
    }
}
verifier, challenge, err := mcpGeneratePKCE()
if err != nil {
    return err
}
state, err := mcpGenerateState()
if err != nil {
    return err
}
port, listener, err := allocLoopbackListener()
if err != nil {
    return err
}
defer listener.Close()
redirectURI := fmt.Sprintf("http://127.0.0.1:%d/callback",
    port)
authURL := mcp.BuildAuthorizationURL(mcp.AuthRequest{
    AuthorizationEndpoint: authEP,
    ClientID:              spec.OAuth.ClientID,
    RedirectURI:           redirectURI,
    Scope:                 spec.OAuth.Scope,
    State:                 state,
    CodeChallenge:         challenge,
})

```

```

    fmt.Fprintf(out, "open this URL in your browser:\n  %s\n",
        authURL)
    _ = openBrowser(authURL)

    code, err := waitForCallback(ctx, listener, state)
    if err != nil {
        return err
    }
    tok, err := mcp.ExchangeCode(ctx, tokEP, code, verifier,
        spec.OAuth.ClientID, redirectURI)
    if err != nil {
        return err
    }
    dir, err := tokenCacheDir()
    if err != nil {
        return err
    }
    tc := &mcp.TokenCache{Dir: dir}
    if err := tc.Save(name, tok); err != nil {
        return err
    }
    fmt.Fprintf(out, "saved token for %s\n", name)
    return nil
}

```



Step 5: Add OAuth helpers (loopback listener, browser opener)

Append to `helix_code/cmd/cli/mcp_cmd.go`:

```

import (
    // merged into existing import block
)

// (in real edit, ensure these imports exist at the top:
//   "encoding/base64"
//   "crypto/rand"
//   "crypto/sha256"
//   "net"
//   "net/http"
//   "net/url"
//   "os/exec"
//   "path/filepath"
//   "runtime"
//   "errors"
// )

func mcpGeneratePKCE() (string, string, error) {
    raw := make([]byte, 48)

```

```

    if _, err := cryptoRandRead(raw); err != nil {
        return "", "", err
    }
    v := base64.RawURLEncoding.EncodeToString(raw)
    sum := sha256.Sum256([]byte(v))
    return v, base64.RawURLEncoding.EncodeToString(sum[:]), nil
}

func mcpGenerateState() (string, error) {
    raw := make([]byte, 24)
    if _, err := cryptoRandRead(raw); err != nil {
        return "", err
    }
    return base64.RawURLEncoding.EncodeToString(raw), nil
}

func cryptoRandRead(p []byte) (int, error) { return
    rand.Read(p) }

func allocLoopbackListener() (int, net.Listener, error) {
    ln, err := net.Listen("tcp", "127.0.0.1:0")
    if err != nil {
        return 0, nil, err
    }
    port := ln.Addr().(*net.TCPAddr).Port
    return port, ln, nil
}

func waitForCallback(ctx context.Context, ln net.Listener,
    wantState string) (string, error) {
    ctx, cancel := context.WithTimeout(ctx, 5*time.Minute)
    defer cancel()
    type result struct {
        code string
        err error
    }
    resCh := make(chan result, 1)
    srv := &http.Server{
        Handler: http.HandlerFunc(func(w http.ResponseWriter, r
            *http.Request) {
            q := r.URL.Query()
            if q.Get("state") != wantState {
                http.Error(w, "state mismatch", 400)
                resCh <- result{err: fmt.Errorf("oauth callback:
state mismatch")}
                return
            }
            if eqe := q.Get("error"); eqe != "" {

```

```

        http.Error(w, ege, 400)
        resCh <- result{err: fmt.Errorf("oauth callback
error: %s", ege)}
        return
    }
    code := q.Get("code")
    fmt.Fprintln(w, "authorization received; you can
close this tab")
    resCh <- result{code: code}
    )),
}
go srv.Serve(ln)
defer srv.Close()
select {
case <-ctx.Done():
    return "", ctx.Err()
case r := <-resCh:
    if r.err != nil {
        return "", r.err
    }
    if r.code == "" {
        return "", errors.New("oauth callback: empty code")
    }
    return r.code, nil
}
}

func openBrowser(u string) error {
    switch runtime.GOOS {
    case "linux":
        return exec.Command("xdg-open", u).Start()
    case "darwin":
        return exec.Command("open", u).Start()
    case "windows":
        return exec.Command("rundll32",
            "url.dll,FileProtocolHandler", u).Start()
    }
    return fmt.Errorf("unsupported platform for browser open")
}

func tokenCacheDir() (string, error) {
    home, err := os.UserConfigDir()
    if err != nil {
        return "", err
    }
    return filepath.Join(home, "helixcode", "mcp", "tokens"), nil
}

```

```
// suppress unused import warnings in case some symbols are not
    yet used
```

```
var _ = url.Parse
```



Step 6: Run cobra tests to verify they pass

```
cd HelixCode && go test -count=1 -run "TestMCPAdd|TestMCPRemove|
    TestMCPList|TestMCPTest" ./cmd/cli/...
```

Expected: PASS (4/4).



Step 7: Write internal/commands/mcp_command.go

Create helix_code/internal/commands/mcp_command.go:

```
package commands
```

```
import (
    "context"
    "fmt"
    "strings"
    "text/tabwriter"
```

```
    "dev.helix.code/internal/mcp"
)
```

```
// MCPCommand implements `/mcp` slash command for the interactive
    CLI.
```

```
type MCPCommand struct {
    manager *mcp.Manager
}
```

```
// NewMCPCommand returns a `/mcp` command bound to a Manager.
```

```
func NewMCPCommand(m *mcp.Manager) *MCPCommand {
    return &MCPCommand{manager: m}
}
```

```
func (c *MCPCommand) Name() string { return "mcp" }
```

```
func (c *MCPCommand) Description() string { return "Manage MCP
    server connections" }
```

```
// Execute runs the slash command. args[0] is the subcommand
    (list|test|logs|reload).
```

```
func (c *MCPCommand) Execute(ctx context.Context, args []string) (
    string, error) {
    if c.manager == nil {
        return "", fmt.Errorf("mcp: manager not initialised")
    }
}
```

```

sub := "list"
if len(args) > 0 {
    sub = args[0]
}
switch sub {
case "list":
    return c.list(), nil
case "test":
    if len(args) < 2 {
        return "", fmt.Errorf("/mcp test <name>")
    }
    if err := c.manager.Test(ctx, args[1]); err != nil {
        return "", err
    }
    return "ready", nil
case "reload":
    cfg := c.manager.Config()
    if cfg == nil {
        return "", fmt.Errorf("/mcp reload: no config loaded")
    }
    if err := c.manager.Reload(ctx, cfg); err != nil {
        return "", err
    }
    return "reloaded", nil
default:
    return "", fmt.Errorf("/mcp: unknown subcommand %q (want list|test|reload)", sub)
}
}

func (c *MCPCCommand) list() string {
    var sb strings.Builder
    tw := tabwriter.NewWriter(&sb, 0, 0, 2, ' ', 0)
    fmt.Fprintln(tw, "NAME\tTRANSPORT\tSTATE\tTOOLS")
    for _, s := range c.manager.Status() {
        fmt.Fprintf(tw, "%s\t%s\t%s\t%d\n", s.Name, s.Transport, s.State, s.ToolCount)
    }
    tw.Flush()
    return sb.String()
}

```



Step 8: Write internal/commands/mcp_command_test.go

Create helix_code/internal/commands/mcp_command_test.go:

```

package commands

import (
    "context"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/mcp"
)

func TestSlashMCP_ListEmpty(t *testing.T) {
    c := NewMCPCommand(mcp.NewManager())
    out, err := c.Execute(context.Background(), nil)
    require.NoError(t, err)
    assert.Contains(t, out, "NAME")
}

func TestSlashMCP_UnknownSubcommand(t *testing.T) {
    c := NewMCPCommand(mcp.NewManager())
    _, err := c.Execute(context.Background(), []string{"nope"})
    assert.Error(t, err)
}

```



Step 9: Register /mcp in builtin/register.go

Edit `helix_code/internal/commands/builtin/register.go` to add (alongside existing `/permissions`, `/worktree`, `/hooks` registrations):

```

// New parameter on RegisterAll: mcpManager *mcp.Manager
// Inside the function, register:
registry.Register(commands.NewMCPCommand(mcpManager))

```

(The exact signature change matches the F02/F04/F05 pattern. Adapt to actual function signature in this file.)



Step 10: Run /mcp slash tests to verify they pass

```

cd HelixCode && go test -count=1 -run "TestSlashMCP" ./internal/
    commands/...

```

Expected: PASS (2/2).



Step 11: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/mcp/ cmd/cli/mcp_cmd.go internal/
commands/mcp_command.go && echo "BLUFF FOUND" || echo
"clean"
```

Expected: clean.



Step 12: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
helix_code/cmd/cli/mcp_cmd.go helix_code/cmd/cli/
mcp_cmd_test.go helix_code/internal/commands/
mcp_command.go helix_code/internal/commands/
mcp_command_test.go helix_code/internal/commands/builtin/
register.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
feat(P1-F06-T11): add helixcode mcp CLI subcommands + /mcp slash
command"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 12: cmd/cli/main.go startup wiring + tools/registry.go integration + integration tests

Files: - Modify: helix_code/cmd/cli/main.go — wire mcp.Manager.Start(ctx) at startup; register mcp cobra command - Modify: helix_code/internal/tools/registry.go — query Manager.Tools() and register external tools - Create: helix_code/tests/integration/mcp_stdio_test.go (real subprocess via test helper echo server) - Create: helix_code/tests/integration/mcp_http_test.go (real httptest.Server, no mocks for the protocol layer)



Step 1: Write the integration test for stdio (TDD)

Create helix_code/tests/integration/mcp_stdio_test.go:

```
//go:build integration
```

```
package integration
```

```
import (
    "context"
    "encoding/json"
    "os"
```

```

"os/exec"
"path/filepath"
"runtime"
"testing"
"time"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"

"dev.helix.code/internal/mcp"
)

func buildEchoBinaryForIT(t *testing.T) string {
    t.Helper()
    dir := t.TempDir()
    bin := filepath.Join(dir, "echo")
    if runtime.GOOS == "windows" {
        bin += ".exe"
    }
    pkg := "../../internal/mcp/testhelper_echo_server"
    out, err := exec.Command("go", "build", "-o", bin,
        pkg).CombinedOutput()
    require.NoError(t, err, string(out))
    return bin
}

// TestMCP_Stdio_FullHandshake exercises Connect → tools/list →
// tools/call → Close
// against a real subprocess. No mocks. Asserts non-empty server
// response.
func TestMCP_Stdio_FullHandshake(t *testing.T) {
    bin := buildEchoBinaryForIT(t)
    cfgDir := t.TempDir()
    cfgPath := filepath.Join(cfgDir, "mcp.yml")
    cfg := &mcp.Config{
        Servers: []mcp.ServerSpec{
            {Name: "echo", Transport: mcp.TransportStdio,
                Command: []string{bin}, AlwaysLoad: true},
        },
    }
    require.NoError(t, mcp.SaveConfig(cfgPath, cfg))

    loaded, err := mcp.LoadConfig(cfgPath)
    require.NoError(t, err)

    mgr := mcp.NewManager()
    mgr.SetConfig(loaded)

```

```

ctx, cancel := context.WithTimeout(context.Background(),
    15*time.Second)
defer cancel()
require.NoError(t, mgr.Start(ctx))
defer mgr.Close()

// runtime evidence: connection actually transitioned to
// ready
deadline := time.Now().Add(5 * time.Second)
for time.Now().Before(deadline) {
    st := mgr.Status()
    if len(st) > 0 && st[0].State == mcp.StateReady {
        break
    }
    time.Sleep(100 * time.Millisecond)
}
st := mgr.Status()
require.Len(t, st, 1)
assert.Equal(t, mcp.StateReady, st[0].State,
    "echo server did not reach ready state")

// echo server replies {} to every request – exercise
// CallTool path
res, err := mgr.CallTool(ctx, "echo", "any",
    map[string]any{"x": 1})
require.NoError(t, err)
require.NotNil(t, res)
b, _ := json.Marshal(res.Raw)
t.Logf("response: %s", b)
_ = os.WriteFile(filepath.Join(cfgDir, "evidence.json"), b,
    0644)
}

```



Step 2: Run integration test to verify the wiring path is exercised

```

cd HelixCode && go test -count=1 -tags=integration -run
    "TestMCP_Stdio_FullHandshake" ./tests/integration/...

```

Expected: PASS.



Step 3: Wire mcp.Manager into cmd/cli/main.go

Edit `helix_code/cmd/cli/main.go` (find the existing startup block where session/hooks/permissions are set up — match that pattern):

1. Add import: `"dev.helix.code/internal/mcp"`
2. After other managers are constructed, add:

```

mcpManager := mcp.NewManager()
{
    userPath := filepath.Join(os.Getenv("HOME"), ".config",
        "helixcode", "mcp.yml")
    projPath := ".helixcode/mcp.yml"
    cfg, err := mcp.LoadMerged(userPath, projPath)
    if err != nil {
        log.Printf("mcp: config load failed: %v (continuing
            without MCP)", err)
    } else {
        mcpManager.SetConfig(cfg)
        if err := mcpManager.Start(ctx); err != nil {
            log.Printf("mcp: start failed: %v", err)
        }
    }
}
defer mcpManager.Close()

```

1. Register the mcp cobra subcommand:

```

rootCmd.AddCommand(newMCPCCommand(MCPCCommandDeps{
    ConfigPath: ".helixcode/mcp.yml",
}))

```

1. Pass mcpManager to wherever commands/builtin.RegisterAll is invoked so / mcp gets registered.
2. Pass mcpManager.Tools() and mcpManager.CallTool into the tools registry.



Step 4: Wire mcp.Manager into internal/tools/registry.go

Edit helix_code/internal/tools/registry.go. Add a method on the existing Registry type:

```

// RegisterMCPManager binds an mcp.Manager so its tools become
// agent-callable.
// Tools are exposed as "<server>:<tool>".
func (r *Registry) RegisterMCPManager(m *mcp.Manager) {
    r.mu.Lock()
    r.mcpManager = m
    r.mu.Unlock()
    r.refreshMCPTools()
}

func (r *Registry) refreshMCPTools() {
    if r.mcpManager == nil {
        return
    }
    for _, t := range r.mcpManager.Tools() {

```

```

name := t.Server + ":" + t.Name
r.Register(&Tool{
    Name:      name,
    Description: t.Desc,
    Schema:    t.Schema,
    Handler: func(ctx context.Context, args
map[string]any) (any, error) {
        res, err := r.mcpManager.CallTool(ctx, t.Server,
t.Name, args)
        if err != nil {
            return nil, err
        }
        return res, nil
    },
})
}
}

```

(Match the actual Tool struct fields and Register signature in the existing file.)



Step 5: Cross-compile sanity (Linux + Windows)

```
cd HelixCode && go build ./... && GOOS=windows go build ./...
```

Expected: both succeed.



Step 6: Full unit test sweep + integration sweep

```
cd HelixCode && go test -count=1 ./internal/mcp/... ./internal/
commands/... ./cmd/cli/...
cd HelixCode && go test -count=1 -tags=integration -run
"TestMCP_" ./tests/integration/...
```

Expected: all PASS.



Step 7: Anti-bluff smoke (broadest scope)

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/mcp/ cmd/cli/main.go cmd/cli/
mcp_cmd.go internal/commands/mcp_command.go internal/
tools/registry.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 8: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/cmd/cli/main.go helix_code/internal/tools/
    registry.go helix_code/tests/integration/
    mcp_stdio_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F06-T12): wire MCP Manager into cmd/cli startup + tools/
    registry + add integration test

Co-Authored-By: Claude Opus 4.7 (1M context)
    <noreply@anthropic.com>

EOF
)"
```

Task 13: Challenge with runtime evidence + cross-compile check

Files: - Create: challenges/p1-f06-mcp-full-lifecycle/CHALLENGE.md - Create: challenges/p1-f06-mcp-full-lifecycle/run.sh - Create: challenges/p1-f06-mcp-full-lifecycle/expected.txt - Modify: docs/improvements/06_phase_1_evidence.md — append runtime evidence



Step 1: Write CHALLENGE.md

Create challenges/p1-f06-mcp-full-lifecycle/CHALLENGE.md:

Challenge: P1-F06 — MCP Full Lifecycle

Purpose

Prove that HelixCode's client-side MCP support actually connects to a real external MCP server, performs the JSON-RPC handshake (initialize → notifications/initialized → tools/list), and successfully invokes a tool. Per Article XI §11.9, every PASS must carry positive runtime evidence.

Procedure

1. Build `bin/helixcode`.`
- 2.

Build the test echo MCP server (a small Go program that speaks MCP over stdio and replies to every request with empty result).

3. Write `.helixcode/mcp.yml`` declaring the echo server with

- ``transport: stdio` and `alwaysLoad: true`.`
4. Run ``helixcode mcp test echo`` – assert exit code 0 and stdout contains "ready".
 5. Run ``helixcode mcp list`` – assert table includes "echo" with transport "stdio".
 6. Anti-bluff smoke: ``grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" helix_code/internal/mcp/`` returns empty.
 7. Cross-compile to Windows: ``G00S=windows go build ./...`` succeeds.

Pass criteria

- Step 4 stdout contains "ready" (no "simulated", no "for now").
- Step 5 stdout contains the server name and transport.
- Step 6 returns clean.
- Step 7 produces a Windows binary.



Step 2: Write run.sh

Create challenges/p1-f06-mcp-full-lifecycle/run.sh:

```
#!/usr/bin/env bash
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT="$(cd "$HERE/../../" && pwd)"
cd "$ROOT/HelixCode"

echo "==> build bin/helixcode"
make build

echo "==> build echo MCP server"
ECHO_BIN="$(mktemp -d)/echo-mcp"
go build -o "$ECHO_BIN" ./internal/mcp/testhelper_echo_server

echo "==> write mcp.yml"
mkdir -p .helixcode
cat > .helixcode/mcp.yml <<EOF
servers:
  - name: echo
    transport: stdio
    command: ["$ECHO_BIN"]
    alwaysLoad: true
EOF

echo "==> helixcode mcp list"
```

```

./bin/helixcode mcp list | tee /tmp/p1f06-list.txt
grep -q "echo" /tmp/p1f06-list.txt
grep -q "stdio" /tmp/p1f06-list.txt

echo "==> helixcode mcp test echo"
./bin/helixcode mcp test echo | tee /tmp/p1f06-test.txt
grep -q "ready" /tmp/p1f06-test.txt
! grep -qi "simulated\\|for now" /tmp/p1f06-test.txt

echo "==> anti-bluff smoke on internal/mcp/"
if grep -rn "simulated\\|for now\\|TODO implement\\|placeholder"
    internal/mcp/; then
    echo "BLUFF FOUND" >&2
    exit 1
fi
echo "clean"

echo "==> cross-compile to windows"
GOOS=windows go build ./...

echo "==> P1-F06 challenge PASS"

```



Step 3: chmod and run

```

chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
    challenges/p1-f06-mcp-full-lifecycle/run.sh
/run/media/milosvasic/DATA4TB/Projects/helix_code/challenges/p1-
    f06-mcp-full-lifecycle/run.sh 2>&1 | tee /tmp/p1f06-
    run.log

```

Expected: terminates with ==> P1-F06 challenge PASS and exit 0.



Step 4: Append runtime evidence to docs/improvements/06_phase_1_evidence.md

Under the F06 section header (already added in T01), append:

T13 – Challenge run

```

```bash
$./challenges/p1-f06-mcp-full-lifecycle/run.sh
==> build bin/helixcode
go build -o bin/helixcode ./cmd/server
==> build echo MCP server
==> write mcp.yml
==> helixcode mcp list
NAME TRANSPORT ALWAYS-LOAD TARGET
echo stdio true /tmp/.../echo-mcp

```

```
==> helixcode mcp test echo
ready
==> anti-bluff smoke on internal/mcp/
clean
==> cross-compile to windows
==> P1-F06 challenge PASS
```

(paste actual terminal output captured during T13 run; no fabrication)

- [ ] **\*\*Step 5: Commit\*\***

```bash

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add challenges/p1-
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit -m "$(cat <
feat(P1-F06-T13): challenge with runtime evidence + cross-compile check
```

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"

Task 14: Feature 6 close-out + push to 4 remotes

Files: - Modify: docs/improvements/PROGRESS.md — flip F06 task list to , advance current focus



Step 1: Update PROGRESS.md current focus

Replace the F06 active block with:

Current focus

```
- **Active phase:** P1 — claude-code feature porting
- **Active feature:** (idle, awaiting next feature pick — F07
  candidate)
- **Active task:** —
-
  **Last completed:** P1-F06-T14 — Feature 6 (MCP Full
  Lifecycle) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none
```



Step 2: Tick all F06 task list items

In the existing P1-F06 task list block, change - [] to - [x] for every item T01–T14.



Step 3: Final smoke + test sweep

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
go test -count=1 ./internal/mcp/... ./internal/commands/... ./
cmd/cli/...
go test -count=1 -tags=integration -run "TestMCP_" ./tests/
integration/...
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder"
internal/mcp/ cmd/cli/mcp_cmd.go internal/commands/
mcp_command.go && echo "BLUFF FOUND" || echo "clean"
go build ./... && GOOS=windows go build ./...
```

Expected: all PASS, clean, both builds succeed.



Step 4: Commit close-out

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
improvements/PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
chore(P1-F06-T14): Feature 6 (MCP Full Lifecycle) close-out

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
EOF
)"
```



Step 5: Push non-force to all 4 remotes (programme convention)

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
origin main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
github main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
gitlab main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
upstream main
```

Expected: each push reports fast-forward; no --force flag anywhere.

Self-review notes (for the implementing agent)

1. **Spec coverage:** every section of the spec has at least one task — types/interface (T02), 4 transports (T03–T06), OAuth (T07), lifecycle (T08), registry (T09), config (T10), CLI + slash (T11), wiring + integration tests (T12), Challenge + cross-compile (T13), close-out (T14).

2. **TDD discipline:** every code-introducing task starts with a failing test (Step 1), runs it to confirm failure (Step 2), implements minimally (Step 3+), reruns to confirm pass.
3. **Type consistency:** `Transport`, `MCPMessage`, `ServerSpec`, `Client`, `Manager` are spelled identically across all tasks. `TransportStdio`/`TransportHTTP`/`TransportSSE`/`TransportWS` are the only transport constants and appear in every relevant task.
4. **Cross-platform:** stdio process-group control split into `transport_stdio_unix.go` and `transport_stdio_windows.go` (T03) — same pattern as F05's `shell_runner_*.go`. Cross-compile check appears in T03, T12, T13, T14.
5. **Anti-bluff:** every task that introduces code runs the FULL 4-term smoke pattern `simulated\|for now\|TODO implement\|placeholder` against `internal/mcp/`. The Challenge in T13 captures real runtime evidence per Article XI §11.9.
6. **No new external dependencies:** `gorilla/websocket`, `gopkg.in/yaml.v3`, `golang.org/x/oauth2`, `github.com/spf13/cobra`, `github.com/stretchr/testify` all already in `go.mod`. `golang.org/x/sys` is implicit via `stdlib` transitions.
7. **Branch + push:** stays on `main`, pushes non-force to all four remotes (`origin/github/gitlab/upstream`) per the programme convention validated in F01–F05.